



南开大学人工智能学院
College Of Artificial Intelligence

计算机组成原理

王明华
南楼214



第一章题目

- 1.课本P6页，例1.2，将平均周期数依次改为2，5，2，5，重新计算该题目。
- 2.课本P16页，课后题4、5、6、7.其中，题4增加：冯诺依曼型计算机的特点是什么？请画出冯诺依曼的硬件框图。

第二章题目

- 1.课本P61页第二章课后题目4-7, 9, 11-13; 第12题为IEEE754 32位浮点数格式
- 2.课后题, 第9问, 除阶符、数符外, 阶码3位, 尾数6位
- 本周末前发到王斌老师邮箱

第2章 运算方法和运算器

2.1 数据与文字的表示方法

2.2 定点加法、减法运算

2.3 定点乘法运算

2.4 定点除法运算

2.5 定点运算器的组成

2.6 浮点运算方法和浮点运算器

2.1 数据与文字的表达方法

2.1.1 数据格式

2.1.2 数的机器码表示

2.1.3 字符与字符串的表示方法

2.1.4 汉字的表示方法

2.1.5 校验码

2.1 数据与文字的表达方法

- 计算机中使用的数据可分成两大类：
 - 符号数据:非数字符号的表示（ASCII、汉字、图形等）
 - 数值数据:数字数据的表示方式（定点、浮点）
- 计算机数字和字符的表示方法应有利于数据的存储、加工(处理)、传送；
- 编码：用少量、简单的基本符号，选择合适的规则表示尽量多的信息，同时利于信息处理（速度、方便）

2.1 数据与文字的表达方法

字 符	ASCII码			字 符	ASCII码		
	十进制K	二进制B	十六进制H		十进制K	二进制B	十六进制H
NUL (空)	0	0	0	M	77	1001101	4D
换行	10	1010	A	N	78	1001110	4E
空格	32	100000	20	O	79	1001111	4F
! (感叹号)	33	100001	21	P	80	1010000	50
"	34	100010	22	Q	81	1010001	51
#	35	100011	23	R	82	1010010	52
\$	36	100100	24	S	83	1010011	53
%	37	100101	25	T	84	1010100	54
&	38	100110	26	U	85	1010101	55
' (引号)	39	100111	27	V	86	1010110	56
(	40	101000	28	W	87	1010111	57
)	41	101001	29	X	88	1011000	58
*	42	101010	2A	Y	89	1011001	59
+	43	101011	2B	Z	90	1011010	5A
,	44	101100	2C	[	91	1011011	5B
- (减号)	45	101101	2D	\	92	1011100	5C
.	46	101110	2E	]	93	1011101	5D
/ (除号)	47	101111	2F	^	94	1011110	5E
0	48	110000	30	-	95	1011111	5F
1	49	110001	31	a	97	1100001	61
2	50	110010	32	b	98	1100010	62
3	51	110011	33	c	99	1100011	63
4	52	110100	34	d	100	1100100	64
5	53	110101	35	e	101	1100101	65
6	54	110110	36	f	102	1100110	66
7	55	110111	37	g	103	1100111	67
8	56	111000	38	h	104	1101000	68
9	57	111001	39	i	105	1101001	69
:	58	111010	3A	j	106	1101010	6A
;	59	111011	3B	k	107	1101011	6B
<	60	111100	3C	l	108	1101100	6C
=	61	111101	3D	m	109	1101101	6D
>	62	111110	3E	n	110	1101110	6E
?	63	111111	3F	o	111	1101111	6F
@	64	1000000	40	p	112	1110000	70
A	65	1000001	41	q	113	1110001	71
B	66	1000010	42	r	114	1110010	72
C	67	1000011	43	s	115	1110011	73
D	68	1000100	44	t	116	1110100	74
E	69	1000101	45	u	117	1110101	75
F	70	1000110	46	v	118	1110110	76
G	71	1000111	47	w	119	1110111	77
H	72	1001000	48	x	120	1111000	78
I	73	1001001	49	y	121	1111001	79
J	74	1001010	4A	z	122	1111010	7A

可分成两大类:

表示 (ASCII、汉字、图

示方式 (定点、浮点)

表示方法应有利于数据
传送;

的基本符号, 选择合适
信息, 同时利于信息处

2.1.1 数据格式

一、定点表示法

- 所有数据的小数点位置固定不变
- 理论上位置可以任意，但实际上将数据表示有两种方法（小数点位置固定-定点表示法/定点格式）：
 - 纯小数
 - 纯整数

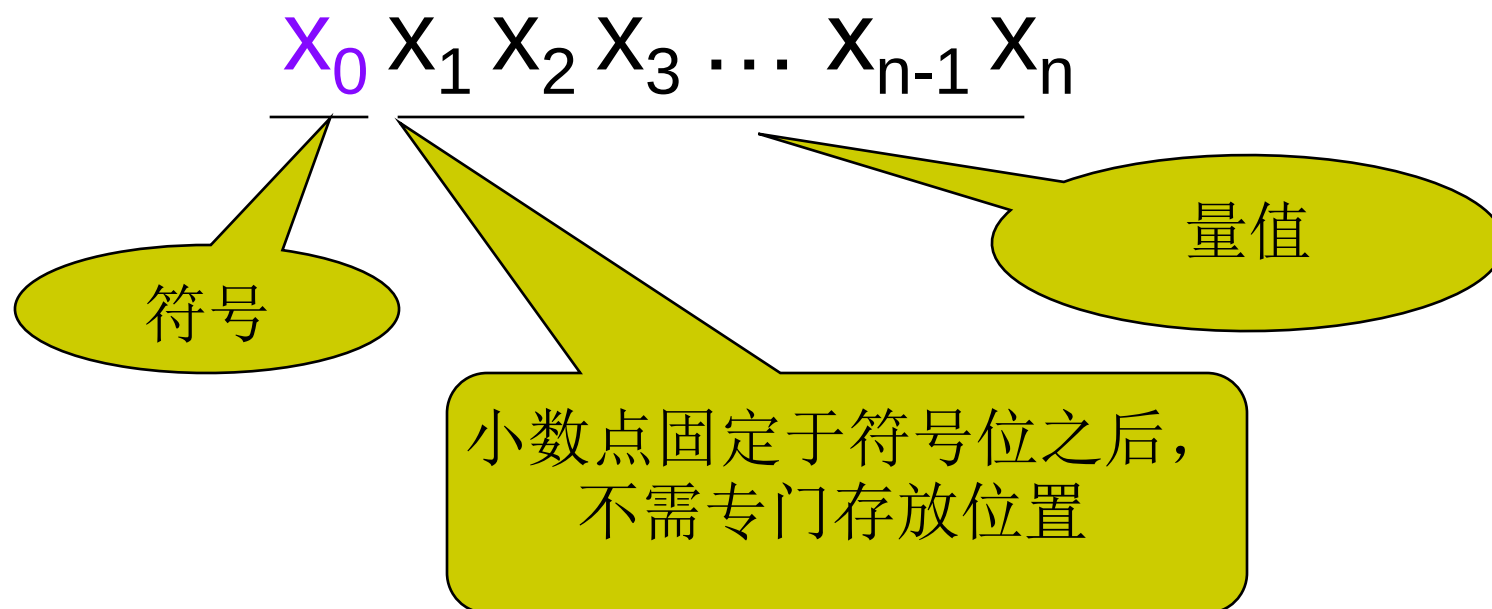
2.1.1 数据格式

例

0	1	0	1
---	---	---	---

+0.101

1. 定点纯小数

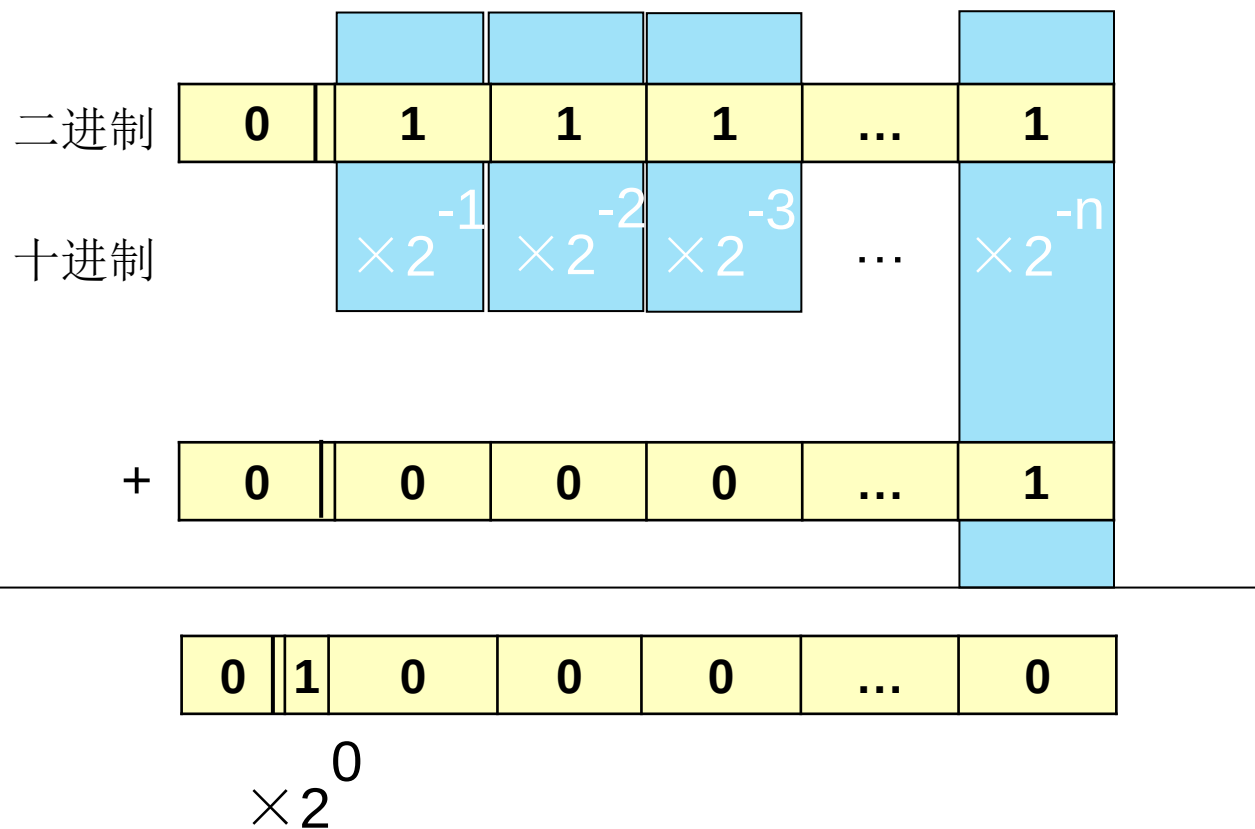


表示范围:

$$1111..111 \leq X \leq 01111..111$$

2.1.1 数据格式

1. 定点纯小数



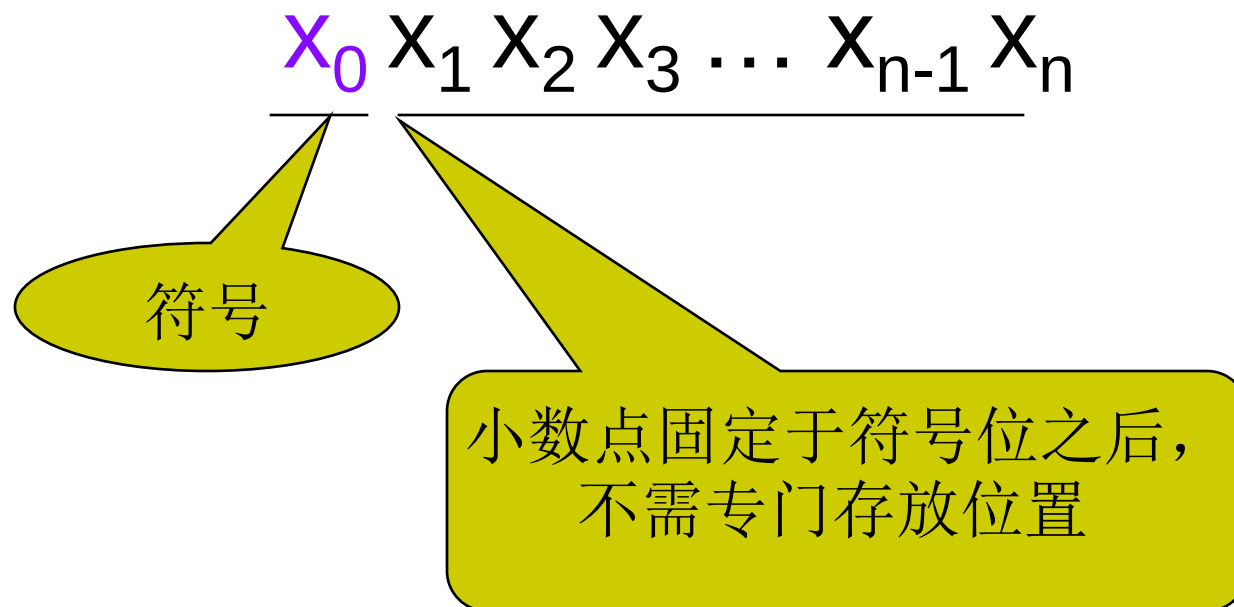
2.1.1 数据格式

例

0	1	0	1
---	---	---	---

+0.101

1. 定点纯小数



表示范围: $-(1-2^{-n}) \leq X \leq (1-2^{-n})$

$$0 \leq |X| \leq (1-2^{-n})$$

2.1.1 数据格式

例

0	1	0	1
---	---	---	---

+ 101

3. 定点纯整数

$x_0 x_1 x_2 x_3 \dots x_{n-1} x_n$

符号

量值

小数点固定于最后一位之后，
不需专门存放位置

2.1.1 数据格式

1. 定点纯整数

$$\begin{array}{c} \times 2^{n-1} \quad \times 2^{n-2} \quad \times 2^{n-3} \quad \dots \quad \times 2^0 \end{array} \bigg| \begin{array}{c} \times 2^{-1} \quad \times 2^{-2} \quad \dots \quad \times 2^{-n} \end{array}$$

二进制	0	1	1	1	...	1
十进制		$\times 2^{n-1}$	$\times 2^{n-2}$	$\times 2^{n-3}$	...	$\times 2^0$
+	0	0	0	0	...	1
						$\times 2^0$

0	1	0	0	0	...	0
---	---	---	---	---	-----	---

$\times 2^n$

2.1.1 数据格式

例

0	1	0	1
---	---	---	---

+ 101

3. 定点纯整数

$X_0 X_1 X_2 X_3 \dots X_{n-1} X_n$

符号

量值

小数点固定于最后一位之后，
不需专门存放位置

表示数的范围是 $-(2^n - 1) \leq X \leq +(2^n - 1)$

$$0 \leq |X| \leq +(2^n - 1)$$

2.1.1 数据格式

4. 定点表示法的特点

- 定点数表示数的范围受字长限制，表示数的范围有限；
- 定点表示的精度有限
- 机器中，常用定点纯整数表示；

问题1:

定点纯小数

0	1	0	1
---	---	---	---

+0.101

定点纯整数

0	1	0	1
---	---	---	---

+ 101

问题2:

0	1		1
---	---	-------	---

2.1.1 数据格式

太阳质量(克): 0.2×10^{34}

电子质量(克): 0.9×10^{-27}

2.1.1 数据格式

5.浮点表示：小数点位置随阶码不同而浮动

1) 格式: $N = R^E.M$

基数R,取固定的值,比如10或2, 隐含表示

指数E

尾数M

其他教材格式: $N = M \times R^E$

R: radix 底, 基数 M: mantissa 小数、尾数

E: exponent 指数、幂次

2.1.1 数据格式

浮点数表示：

$$(1.75)_{10} =$$

十进制转二进制规则：

整数：“除2取余，逆序排列”

小数：“乘2取整，顺序排列”

2.1.1 数据格式

浮点数表示:

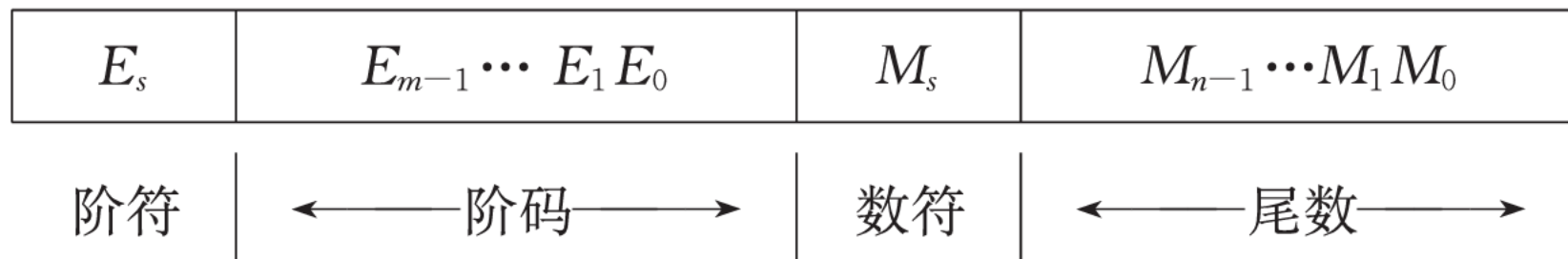
$$\begin{aligned}(1.75)_{10} &= 1.11 \times 2^0 \\ &= 11.1 \times 2^{-1} \quad \text{二进制} \\ &= 111 \times 2^{10} \\ \checkmark &= 0.111 \times 2^1 \quad \text{二进制} \\ &= 0.0111 \times 2^{10}\end{aligned}$$

阶码, 整数, 可正可负
尾数, 小数, 可正可负

2.1.1 数据格式

5.浮点表示：小数点位置随阶码不同而浮动

2) 机器中表示



- ✓ 尾数M: 决定浮点数的表示精度
- ✓ 指数\阶码E: 决定浮点数的表示范围

2.1.2 数的机器码表示

- 真值：一般书写的数
- 机器码：机器中表示的数, 要解决在计算机内部数的正、负符号和小数点运算问题。
 - 原码
 - 反码
 - 补码
 - 移码

1.原码表示法

● 定点整数 $X_0X_1X_2\ldots X_n$

$$[x]_{\text{原}} = \begin{cases} x & 2^n > x \geq 0 \\ 2^n - x & 0 \geq x > -2^n \end{cases} \quad \text{符号} \quad \begin{cases} 0, \text{ 正数} \\ 1, \text{ 负数} \end{cases}$$

说明:

- 有正0和负0之分
- 范围 $-(2^n-1) \sim +(2^n-1)$
- 例: $x=+11001110$, $y=-11001110$
 $[x]_{\text{原}}=011001110$ $[y]_{\text{原}}=111001110$

1.原码表示法

- 定点小数 $x_0.x_1x_2\dots x_n$

$$[x]_{\text{原}} = \begin{cases} x & 1 > x \geq 0 \\ 1-x & 0 \geq x > -1 \end{cases} \quad \text{符号} \begin{cases} 0, \text{ 正} \\ 1, \text{ 负数} \end{cases}$$

- 有正0和负0之分
- 范围 $-(1-2^{-n}) \sim +(1-2^{-n})$

例: $x=+0.11001110$, $y=-0.11001110$

$[x]_{\text{原}}=0.11001110$ $[y]_{\text{原}}=1.11001110$

1.原码表示法

原码特点:

- 表示简单，易于同真值之间进行转换，实现乘除运算规则简单。
- 进行加减运算十分麻烦。

2.1.2 数的机器码表示

2.补码表示法

- 生活例子：现为北京时间下午4点，但钟表显示为7点。有两种办法校对：
 - (1) 做减法 $7-3 = 4$ (逆时针退3格)
 - (2) 做加法 $7+9 = 16$ (顺时针进9格)
 $16 \pmod{12} = 16-12 = 4$ (以12为模，变成4)

2.补码表示法

- 定义：正数的补码就是正数的本身，负数的补码是原负数加上模。
- 计算机运算受字长限制,属于有模运算。
 - 定点小数 $x_0.x_1x_2\dots x_n$ ，以2为模
 - 定点整数 $x_0.x_1x_2\dots x_n$ ，以 2^{n+1} 为模
- 定点小数 $x_0.x_1x_2\dots x_n$

$$[x]_{\text{补}} = \begin{cases} x & 1 > x \geq 0 \\ 2+x & 0 > x \geq -1 \end{cases} \quad \text{符号} \begin{cases} 0, \text{ 正小数} \\ 1, \text{ 负小数} \end{cases}$$

2.补码表示法

- 定点整数 $x_0.x_1x_2\dots x_n$

$$[x]_{\text{补}} = \begin{cases} x & 2^n > x \geq 0 \\ 2^{n+1} + x & 0 > x \geq -2^n \end{cases} \quad \text{符号} \begin{cases} 0, \text{ 正整数} \\ 1, \text{ 负整数} \end{cases}$$

补码最大的优点就是将减法运算转换成加法运算。
通常不按表达式求补码，而通过反码来得到。

求补码的快捷方式

设 $x = -1010$ 时

$$\begin{aligned}
 \text{则 } [x]_{\text{补}} &= 2^{4+1} - 1010 = 11111 + 1 - 1010 \\
 &= 100000 = 11111 + 1 \\
 &\quad - 1010 \\
 \hline
 &= 1,0110
 \end{aligned}
 \qquad
 \begin{aligned}
 &= 11111 + 1 \\
 &\quad - 1010 \\
 \hline
 &= \boxed{10101} + 1 \\
 &= 1,0110
 \end{aligned}$$

又 $[x]_{\text{原}} = \boxed{1,1010}$

当真值为 负 时，补码 可用 原码除符号位外
每位取反，末位加 1 求得

3.反码表示法

- 定义：正数的表示与原、补码相同，负数的补码符号位为1，数值位是将原码的数值按位取反，就得到该数的反码表示。
- 电路容易实现，触发器的输出有正负之分。

3.反码表示法

- 对尾数求反，它跟补码的区别在于末位少加一个1，所以可以推出反码的定义
 - 定点整数 $x_0.x_1x_2...x_n$

$$[x]_{\text{反}} = \begin{cases} x & 2^n > x \geq 0 \\ 2^{n+1} + x - 1 & 0 \geq x > -2^n \end{cases}$$

- 反码表示有正0和负0之分

3.反码表示法

- 对尾数求反，它跟补码的区别在于末位少加一个1，所以可以推出反码的定义
 - 定点小数 $x_0.x_1x_2...x_n$

$$[x]_{\text{反}} = \begin{cases} x & 1 > x \geq 0 \\ 2+x-2^{-n} & 0 \geq x > -1 \end{cases}$$

例：

$X_1 = +0.1011011$, $[X_1]_{\text{反}} = 0.1011011$

$X_2 = -0.1011011$, $[X_2]_{\text{反}} = 1.0100100$

补码求原码

例 6.5 已知 $[x]_{\text{补}} = 0.0001$

求 x

解：由定义得 $x = + 0.0001$

例 6.6 已知 $[x]_{\text{补}} = 1.0001$ $[x]_{\text{补}} \xrightarrow{?} [x]_{\text{原}}$

求 x

$$[x]_{\text{原}} = 1.1111$$

解：由定义得

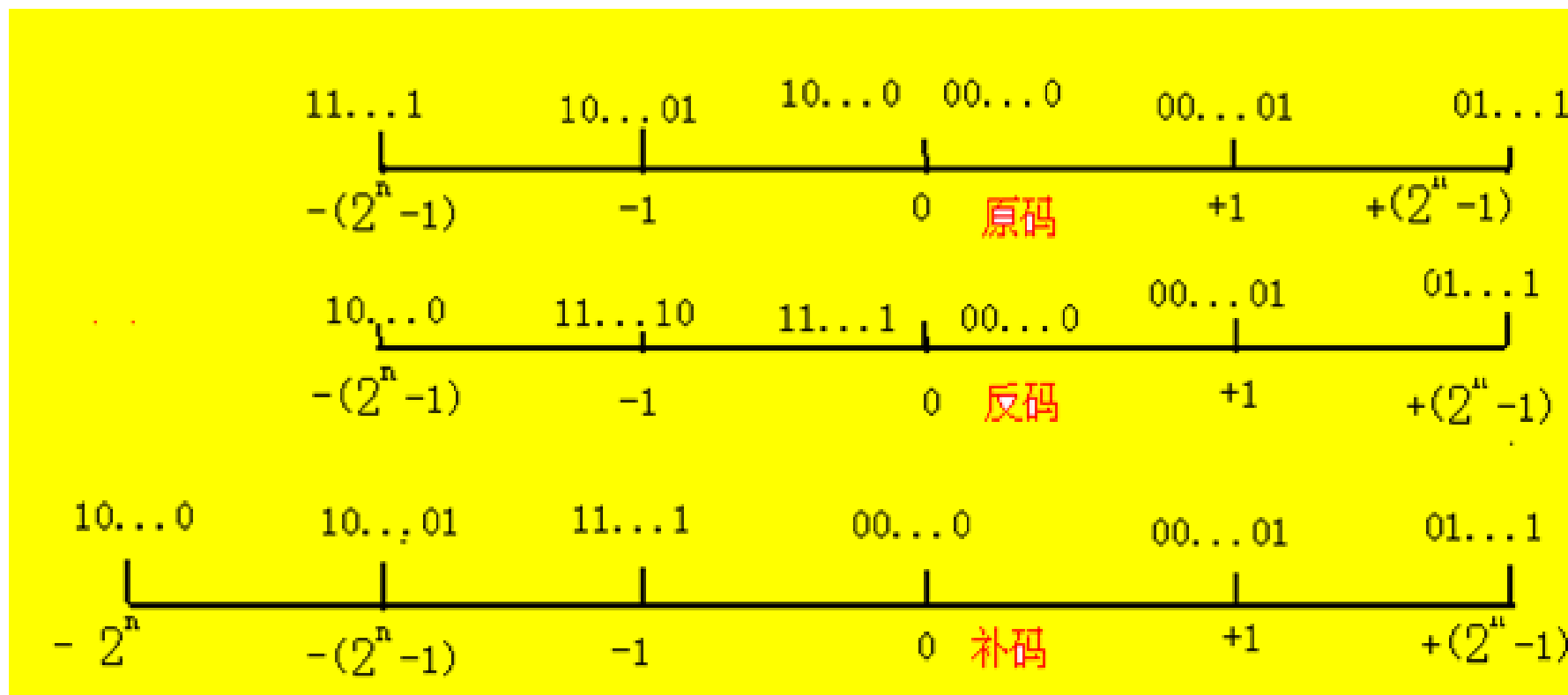
$$\therefore x = -0.1111$$

$$\begin{aligned} x &= [x]_{\text{补}} - 2 \\ &= 1.0001 - 10.0000 \\ &= -0.1111 \end{aligned}$$

当真值为 负 时，原码 可用 补码除符号位外

每位取反，末位加 1 求得

[例6]以定点整数为例,用数轴形式说明原码、反码、补码表示范围和可能的数码组合情况。



[例7]将十进制真值(-127,-1,0,+1,+127)列表表示成二进制数及原码、反码、补码、移码值。

真值x (十进制)	真值x (二进制)	[x]原	[x]反	[x]补
-127	-01111111	11111111	10000000	10000001
-1	-00000001	10000001	11111110	11111111
		00000000	00000000	
0	00000000			00000000
		10000000	11111111	
+1	+00000001	00000001	00000001	00000001
+127	+01111111	01111111	01111111	01111111

补码表示很难直接判断其真值大小

4.移码表示法

- 移码表示法（用在阶码中）
 - 定点整数定义 $[x]_{\text{移}} = 2^n + x \quad 2^n > x \geq -2^n$
 - 00000000~11111111($-2^n \sim 2^n - 1$)
 - 例1 $x = +1011111$
原码为01011111
补码为01011111
反码为01011111
移码为11011111

4.移码表示法

例2: $x = -1011111$

原码为11011111

补码为10100001

反码为10100000

移码为00100001

特点: 移码和补码尾数相同, 符号位相反

[例7]将十进制真值(-127,-1,0,+1,+127)列表表示成二进制数及原码、反码、补码、移码值。

真值x (十进制)	真值x (二进制)	[x]原	[x]反	[x]补	[x]移
-127	-01111111	11111111	10000000	10000001	00000001
-1	-00000001	10000001	11111110	11111111	01111111
		00000000	00000000		
0	00000000			00000000	10000000
		10000000	11111111		
+1	+00000001	00000001	00000001	00000001	10000001
+127	+01111111	01111111	01111111	01111111	11111111

[例8]设机器字长16位,定点表示,尾数15位

(1)定点原码整数表示时,最大正数是多少?最小负数是多少?

0 111 111 111 111 111 最大正整数

$$x = (2^{15} - 1)_{10} = (+32767)_{10}$$

1 111 111 111 111 111 最小负整数

$$x = (1 - 2^{15})_{10} = -(2^{15} - 1)_{10} = (-32767)_{10}$$

(2)定点原码小数表示, 最大正数是多少?最小负数是多少?

0 111 111 111 111 111 最大正小数

$$x = (1 - 2^{15})_{10}$$

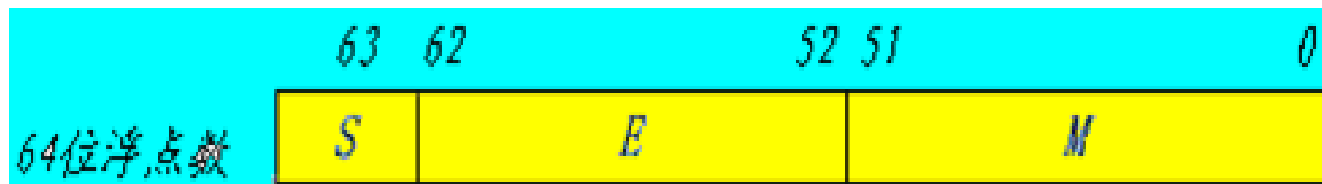
1 111 111 111 111 111 最小负小数

$$x = -(1 - 2^{15})_{10}$$

2.1.1 数据格式

6. IEEE754标准(规定了浮点数的表示格式,运算规则等)

- 规则规定了单精度(32)和双精度(64)的基本格式。
- 规则中,尾数用原码,指数用移码(便于对阶和比较)



2.1.1 数据格式

IEEE754标准

- 基数 $R=2$ ，基数固定，采用隐含方式来表示它。
- 32位的浮点数：
 - S数的符号位，1位，在最高位，“0”表示正数，“1”表示负数。
 - M是尾数，23位，在低位部分，采用原码小数表示
 - E是阶码，8位，阶符采用隐含方式。采用移码表示。移码比较大方便。
 - 规格化：若不对浮点数的表示作出明确规定，同一个浮点数的表示就不是唯一的。
 - 尾数域最左位(最高有效位)总是1，故这一位经常不予存储，而认为隐藏在小数点的左边。
 - 采用这种方式时，将浮点数的指数真值 e 变成阶码 E 时，应将指数 e 加上一个固定的偏移值127(01111111)，即 $E=e+127$ 。



2.1.1 数据格式

- E为什么偏移127?

2.1.1 数据格式

- 真值 x 为零表示：当阶码 E 为全0且尾数 M 也为全0时的值，结合符号位 S 为0或1，有正零和负零之分。（机器零）
- 真值 x 为无穷大表示：当阶码 E 为全1且尾数 M 为全0时，结合符号位 S 为0或1，也有 $+\infty$ 和 $-\infty$ 之分。
- 这样在32位浮点数表示中，要除去 E 用全0和全1（ 255_{10} ）表示零和无穷大的特殊情况，指数的偏移值不选128（10000000），而选127（01111111）。对于规格化浮点数， E 的范围变为1到254，真正的指数值 e 则为-126到+127。因此32位浮点数表示的绝对值的范围是 $10^{-38} \sim 10^{38}$ （以10的幂表示）。
- 浮点数所表示的范围远比定点数大。一台计算机中究竟采用定点表示还是浮点表示，要根据计算机的使用条件来确定。一般在高档微机以上的计算机中同时采用定点、浮点表示，由使用者进行选择。而单片机中多采用定点表示。

2.1.1 数据格式

- 一个规格化的32位浮点数x的真值表示为

$$x=(-1)^S \times (1.M) \times 2^{E-127}$$

$$e=E-127$$

- 64位的浮点数中符号位1位，阶码域11位，尾数域52位，指数偏移值是1023。因此规格化的64位浮点数x的真值为：

$$x=(-1)^S \times (1.M) \times 2^{E-1023}$$

$$e=E-1023$$

2.1.1 数据格式

例1：若浮点数x的754标准存储格式为 $(41360000)_{16}$ ，求其浮点数的十进制数值。

解：将16进制数展开后，可得二制数格式为

0 100 00010 011 0110 0000 0000 0000 0000

符号S 阶码E(8位) 尾数M(23位)

指数 $e = E - 127 = 10000010 - 01111111 = 00000011 = (3)_{10}$

包括隐藏位1的尾数

$1.M = 1.011\ 0110\ 0000\ 0000\ 0000\ 0000 = 1.011011$

于是有

$x = (-1)^S \times 1.M \times 2^e = +(1.011011) \times 2^3 = +1011.011 = (11.375)_{10}$

2.1.1 数据格式

例2：将数 $(20.59375)_{10}$ 转换成754标准的32位浮点数的二进制和十六进制存储格式。

解：

十进制转二进制规则：

整数：“除2取余，逆序排列”

小数：“乘2取整，顺序排列”

2.1.1 数据格式

例2：将数 $(20.59375)_{10}$ 转换成754标准的32位浮点数的二进制存储格式。

解：首先分别将整数和分数部分转换成二进制数：

$$20.59375 = 10100.10011$$

然后移动小数点，使其在第1，2位之间

$$10100.10011 = 1.010010011 \times 2^4$$

$e=4$ 于是得到：

$$S=0, E=4+127=131, M=010010011$$

最后得到32位浮点数的二进制存储格式为：

$$01000001101001001100000000000000 = (41A4C000)_{16}$$

例9 假设由**S,E,M**三个域组成的一个**32**位二进制字所表示的非零规格化浮点数 x ,真值表示为（注意此例非**IEEE754**标准）：

$$x = (-1)^S \times (1.M) \times 2^{E-128}$$

问：它所表示的规格化的最大正数、最小正数、最大负数、最小负数是多少？

(1)最大正数

0 11 111 111 111 111 111 111 111 111 111 111 11 11

$$x = [1 + (1 - 2^{-23})] \times 2^{127}$$

(2)最小正数

0 00 000 000 000 000 000 000 000 000 000 000 00 00

$$x = 1.0 \times 2^{-128}$$

(3)最小负数

1 11 111 111 111 111 111 111 111 111 111 111 11 11

$$x = -[1 + (1 - 2^{-23})] \times 2^{127}$$

(4)最大负数

1 00 000 000 000 000 000 000 000 000 000 000 00 00

$$x = -1.0 \times 2^{-128}$$

$$N = R^E.M$$

2.1.3 字符与字符串的表示方法

- 符号数据：字符信息用数据表示，如ASCII等；
- 字符表示方法ASCII:用一个字节来表示,低7位用来编码(128),最高位为校验位,参见教材P24表2.1.3
- 字符串的存放方法
 - 每个字节存一个字符

2.1.4 汉字的表示方法

- 汉字的表示方法

（一级汉字3755个，二级汉字3008个）

- 输入码

- 国标码

- 一级（16~55）*94
- 二级（56~87）*94
- 图形符号（682个）（01~09）*94

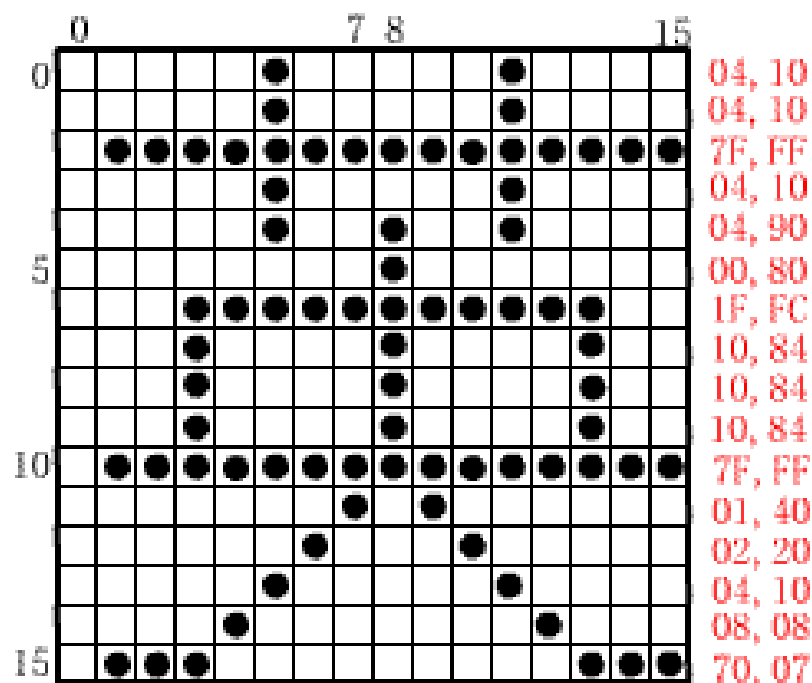
- 拼音、五笔

- 汉字内码：汉字信息的存储，交换和检索的机内代码，两个字节组成，每个字节高位都为1（区别于英文字符）

2.1.4 汉字的表示方法

- 汉字字模码：汉字字形

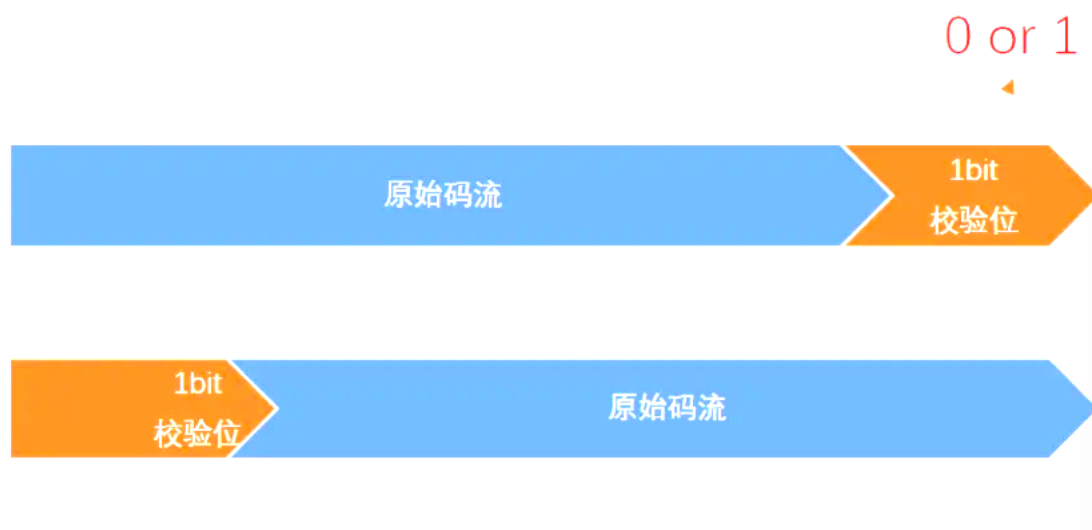
- 点阵



- 汉字库

2.1.5 校验码

- 校验码（只介绍奇偶校验码）
 - 奇校验：原始码流+校验位，添加校验位使得总共有奇数个1
 - 偶校验：原始码流+校验位，添加校验位使得总共有偶数个1



2.1.5 校验码

原始码

奇校验

偶校验

奇数个1

偶数个1

1011000

10110000

10110001

1010000

10100001

10100000

0011010

00110100

00110101

0001000

00010000

00010001

0000000

00000001

00000000

2.1.5 校验码

- 只能检测出奇数个错误。

举例：ASCII码 大写字母 A

十六进制 0x41

二进制 01000001 → 有效位为低7位

奇校验 11000001

偶校验 01000001

最高位是奇偶校验位

奇校验 正确码流 11000001

错1位 11000011 变成了偶数个1，能检测出错误

错2位 11000010 变成了奇数个1，检测不出错误

错3位 11001010 变成了偶数个1，能检测出错误

偶校验 正确码流 01000001

错1位 01000011 变成了奇数个1，能检测出错误

错2位 01000010 变成了偶数个1，检测不出错误

错3位 01001010 变成了奇数个1，能检测出错误



2.2 定点加法、减法运算

2.2.1 补码加法

2.2.2 补码减法

2.2.3 溢出概念与检测方法

2.2.4 基本的二进制加法/减法器

2.2.1 补码加法

- 补码加法

公式: $[x+y]_{\text{补}} = [x]_{\text{补}} + [y]_{\text{补}} \pmod{2^{n+1}}$

$[x]_{\text{补}} + [y]_{\text{补}} = [x + y]_{\text{补}}$ 证明

- P28（了解即可）

2.2.1 补码加法

- [例11] $x=+1011$, $y=+0101$, 求 $x+y=?$

解: $[x]_{\text{补}} = 01001$, $[y]_{\text{补}} = 00101$

$$\begin{array}{r}
 \phantom{[x]_{\text{补}}} \phantom{[y]_{\text{补}}} \\
 + \phantom{[x]_{\text{补}}} \phantom{[y]_{\text{补}}} \\
 \hline
 \phantom{[x]_{\text{补}}} \phantom{[y]_{\text{补}}}
 \end{array}$$

$\therefore x+y = +1110$

2.2.1 补码加法

[例12] $x=+1011$, $y=-0101$, 求 $x+y=?$

解: $[x]_{\text{补}} = 01001$, $[y]_{\text{补}} = 11011$

$$\begin{array}{r}
 \phantom{[x]_{\text{补}}} \phantom{[y]_{\text{补}}} \\
 + \phantom{[x]_{\text{补}}} \phantom{[y]_{\text{补}}} \\
 \hline
 [x+y]_{\text{补}}
 \end{array}$$

$\therefore x+y = +0110$

2.2.2 补码减法

公式:

$$[x-y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}} = [x]_{\text{补}} - [y]_{\text{补}}$$

$$[-y]_{\text{补}} = -[y]_{\text{补}} \quad (\text{mod } 2^{n+1})$$

运算法则：带符号位求反，最末位+1

2.2.2 补码减法

[例13] 已知 $x_1 = -1110$ ， $x_2 = +1101$ ，求：

$[x_1]_{\text{补}}$ ， $[-x_1]_{\text{补}}$ ， $[x_2]_{\text{补}}$ ， $[-x_2]_{\text{补}}$ 。

解：

$$[x_1]_{\text{补}} = 10010$$

$$[-x_1]_{\text{补}} = -[x_1]_{\text{补}} + 2^{-4} = 01101 + 00001 = 01110$$

$$[x_2]_{\text{补}} = 01101$$

$$[-x_2]_{\text{补}} = -[x_2]_{\text{补}} + 2^{-4} = 10010 + 00001 = 10011$$

2.2.2 补码减法

[例14] $x=+1101$, $y=+0110$, 求 $[x-y]_{\text{补}}=?$ $x-y=?$

解: $[x]_{\text{补}} = 01101$

$[y]_{\text{补}} = 00110$, $[-y]_{\text{补}} = 11010$

$$\begin{array}{r}
 + \quad [x]_{\text{补}} \quad \quad 0 \ 1 \ 1 \ 0 \ 1 \\
 \quad [-y]_{\text{补}} \quad \quad 1 \ 1 \ 0 \ 1 \ 0 \\
 \hline
 [x-y]_{\text{补}} \quad \quad \underline{1} \ 0 \ 0 \ 1 \ 1 \ 1
 \end{array}$$

$\therefore x-y = +0111$

2.2.3 溢出概念与检测方法

[例15] $x=+1101$, $y=+1001$, 求 $x+y$ 。

解: $[x]_{\text{补}}=01011$, $[y]_{\text{补}}=01001$

$$\begin{array}{r}
 \phantom{[x]_{\text{补}}} \phantom{[y]_{\text{补}}} \phantom{[x+y]_{\text{补}}} \\
 [x]_{\text{补}} \phantom{[y]_{\text{补}}} \phantom{[x+y]_{\text{补}}} \phantom{[x+y]_{\text{补}}} \\
 + \phantom{[x]_{\text{补}}} [x]_{\text{补}} \phantom{[y]_{\text{补}}} \phantom{[x+y]_{\text{补}}} \phantom{[x+y]_{\text{补}}} \\
 \hline
 [x+y]_{\text{补}} \phantom{[y]_{\text{补}}} \phantom{[x+y]_{\text{补}}} \phantom{[x+y]_{\text{补}}}
 \end{array}
 \begin{array}{r}
 0 \ 1 \ 0 \ 1 \ 1 \\
 0 \ 1 \ 0 \ 0 \ 1 \\
 \hline
 1 \ 0 \ 1 \ 0 \ 0
 \end{array}$$

两个正数相加的结果成为负数，表示正溢。

2.2.3 溢出概念与检测方法

[例16] $x=-1101$, $y=-1011$, 求 $x+y$ 。

解: $[x]_{\text{补}}=10011$, $[y]_{\text{补}}=10101$

$$\begin{array}{r}
 + \quad [x]_{\text{补}} \quad 1\ 0\ 0\ 1\ 1 \\
 \quad [y]_{\text{补}} \quad 1\ 0\ 1\ 0\ 1 \\
 \hline
 [x+y]_{\text{补}} \quad 0\ 1\ 0\ 0\ 0
 \end{array}$$

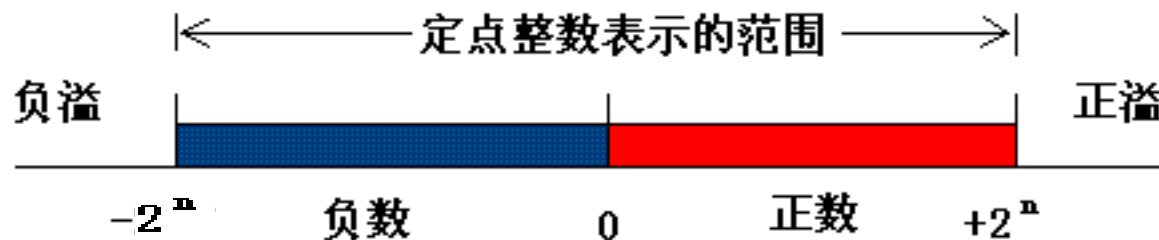
两个负数相加的结果成为正数，表示负溢。

2.2.3 溢出概念与检测方法

- 溢出的概念

- 可能产生溢出的情况

- 两正数加，变负数，正溢（大于机器所能表示的最大数）
 - 两负数加，变正数，负溢（小于机器所能表示的最小数）



2.2.3 溢出概念与检测方法

检测方法

1、双符号位法（变形补码）

$$[x]_{\text{补}} = 2^{n+2} + x \quad (\text{mod } 2^{n+2})$$

2.2.3 溢出概念与检测方法

[例17] $x=+1100$, $y=+1000$, 求 $x+y$ 。

解: $[x]_{\text{补}} = 001100$, $[y]_{\text{补}} = 001000$

$$\begin{array}{r}
 + \quad \begin{array}{l} [x]_{\text{补}} \\ [y]_{\text{补}} \end{array} \quad \begin{array}{l} 001100 \\ 001000 \end{array} \\
 \hline
 \begin{array}{l} [x+y]_{\text{补}} \end{array} \quad \begin{array}{l} 010100 \end{array} \quad (\text{表示正溢})
 \end{array}$$

2.2.3 溢出概念与检测方法

[例18] $x=-1100$, $y=-1000$, 求 $x+y$ 。

解: $[x]_{\text{补}} = 110100$, $[y]_{\text{补}} = 111000$

$$\begin{array}{r}
 + \quad \begin{array}{l} [x]_{\text{补}} \\ [y]_{\text{补}} \end{array} \quad \begin{array}{l} 1\ 1\ 0\ 1\ 0\ 0 \\ 1\ 1\ 1\ 0\ 0\ 0 \end{array} \\
 \hline
 \begin{array}{l} [x+y]_{\text{补}} \end{array} \quad \begin{array}{l} 1\ 0\ 1\ 1\ 0\ 0 \end{array} \quad (\text{表示负溢})
 \end{array}$$

2.2.3 溢出概念与检测方法

检测方法

1、双符号位法（变形补码）

$$[x]_{\text{补}} = 2^{n+2} + x \quad (\text{mod } 2^{n+2})$$

$S_{f1} \quad S_{f2}$

0 0 正确（正数）

0 1 正溢

1 0 负溢

1 1 正确（负数）



- ① S_{f1} 表示正确的符号，逻辑表达式为 $V = S_{f1} \oplus S_{f2}$ ，可以用异或门来实现
- ② 最高符号位为正确符号位。

2.2.3 溢出概念与检测方法

2. 单符号位法

当最高有效位产生进位而符号位无进位时，产生正溢；当最高有效位无进位而符号位有进位时，产生负溢。

- | | | |
|-------|-------|--------|
| C_f | C_0 | |
| 0 | 0 | 正确（正数） |
| 0 | 1 | 正溢 |
| 1 | 0 | 负溢 |
| 1 | 1 | 正确（负数） |



- $V = C_f \oplus C_0$ ，其中 C_f 为符号位产生的进位， C_0 为最高有效位产生

2.2.3 溢出概念与检测方法

[例15] $x=+1101$, $y=+1001$, 求 $x+y$ 。

解: $[x]_{\text{补}}=01011$, $[y]_{\text{补}}=01001$

$$\begin{array}{r} \phantom{[x]_{\text{补}}} \phantom{[y]_{\text{补}}} \phantom{[x+y]_{\text{补}}} \\ [x]_{\text{补}} \phantom{[y]_{\text{补}}} \phantom{[x+y]_{\text{补}}} \phantom{[x+y]_{\text{补}}} \phantom{[x+y]_{\text{补}}} \phantom{[x+y]_{\text{补}}} \\ + \phantom{[x]_{\text{补}}} [x]_{\text{补}} \phantom{[y]_{\text{补}}} \phantom{[x+y]_{\text{补}}} \phantom{[x+y]_{\text{补}}} \phantom{[x+y]_{\text{补}}} \phantom{[x+y]_{\text{补}}} \\ \hline [x+y]_{\text{补}} \phantom{[y]_{\text{补}}} \phantom{[x+y]_{\text{补}}} \phantom{[x+y]_{\text{补}}} \phantom{[x+y]_{\text{补}}} \phantom{[x+y]_{\text{补}}} \end{array}$$

两个正数相加的结果成为负数，表示正溢。

2.2.3 溢出概念与检测方法

[例16] $x=-1101$, $y=-1011$, 求 $x+y$ 。

解: $[x]_{\text{补}}=10011$, $[y]_{\text{补}}=10101$

$$\begin{array}{r}
 + \quad [x]_{\text{补}} \quad 1\ 0\ 0\ 1\ 1 \\
 \quad [y]_{\text{补}} \quad 1\ 0\ 1\ 0\ 1 \\
 \hline
 [x+y]_{\text{补}} \quad 0\ 1\ 0\ 0\ 0
 \end{array}$$

两个负数相加的结果成为正数，表示负溢。

2.2.3 溢出概念与检测方法

2. 单符号位法

当最高有效位产生进位而符号位无进位时，产生正溢；当最高有效位无进位而符号位有进位时，产生负溢。

- | C_f | C_0 | |
|-------|-------|--------|
| 0 | 0 | 正确（正数） |
| 0 | 1 | 正溢 |
| 1 | 0 | 负溢 |
| 1 | 1 | 正确（负数） |



- $V = C_f \oplus C_0$ ，其中 C_f 为符号位产生的进位， C_0 为最高有效位产生

2.2.4 基本的二进制加法/减法器

一位全加器真值表

输入			输出	
A_i	B_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

FA逻辑方程：

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_i B_i + B_i C_i + C_i A_i = A_i B_i + (A_i \oplus B_i) C_i$$

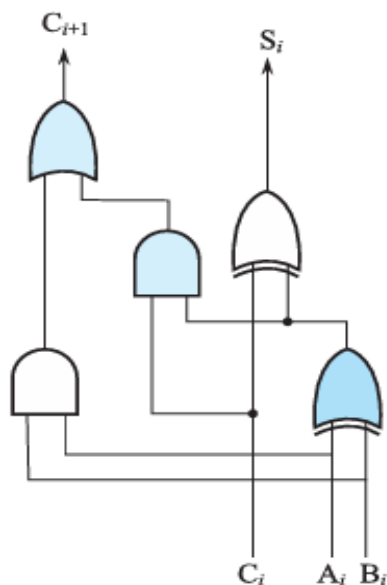
2.2.4 基本的二进制加法/减法器

FA逻辑电路和框图

FA逻辑方程：

$$S_i = A_i \oplus B_i \oplus C_i$$

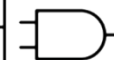
$$C_{i+1} = A_i B_i + B_i C_i + C_i A_i = A_i B_i + (A_i + B_i) C_i$$



FA（全加器）逻辑电路图

● 1.与门

输入		输出
A	B	Q
1	1	1
1	0	0
0	1	0
0	0	0



● 2.或门

输入		输出
A	B	Q
1	1	1
1	0	1
0	1	1
0	0	0



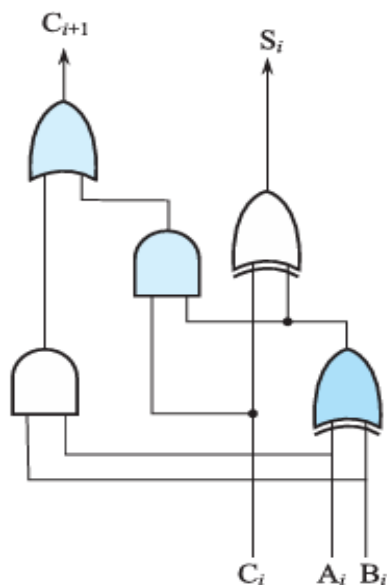
2.2.4 基本的二进制加法/减法器

FA逻辑电路和框图

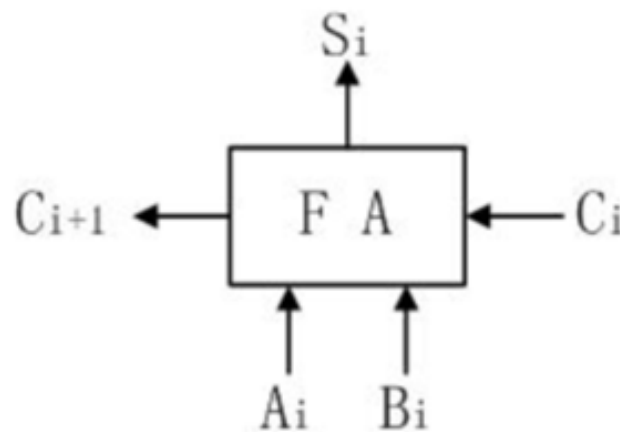
FA逻辑方程：

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_i B_i + B_i C_i + C_i A_i = A_i B_i + (A_i + B_i) C_i$$

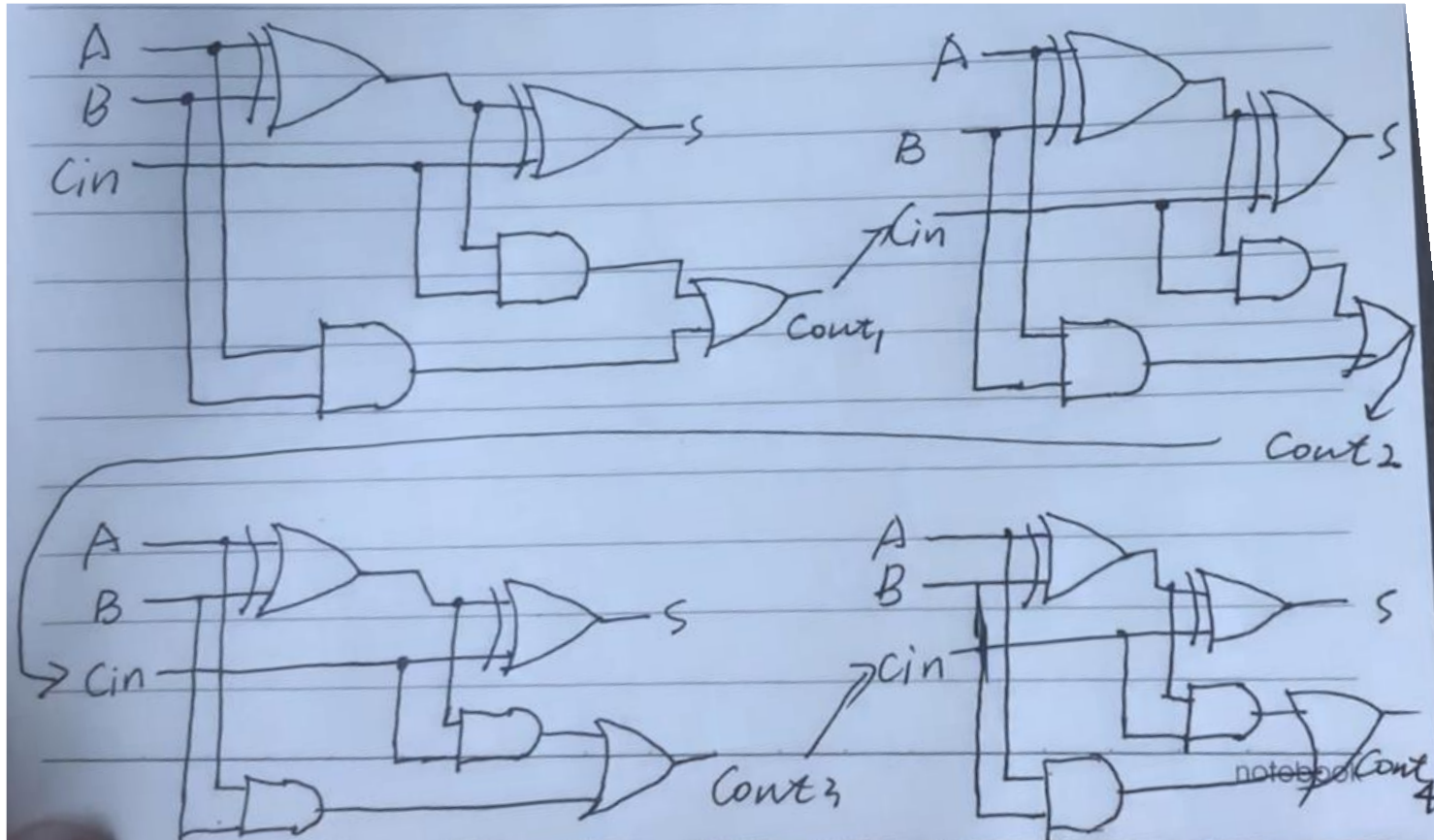


FA（全加器）逻辑电路图



FA框图

4bit-串行进位加减法器



进位位C的延迟时间是 $3T + n \cdot 2T$
求和S的延迟时间是 $3T + (n-1) \cdot 2T + 3T$

2.2.4 基本的二进制加法/减法器

n位行波进位加法器

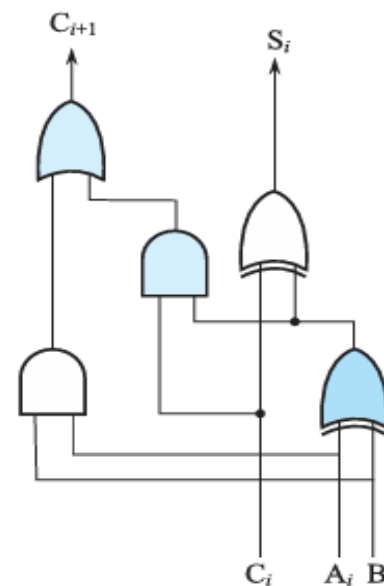
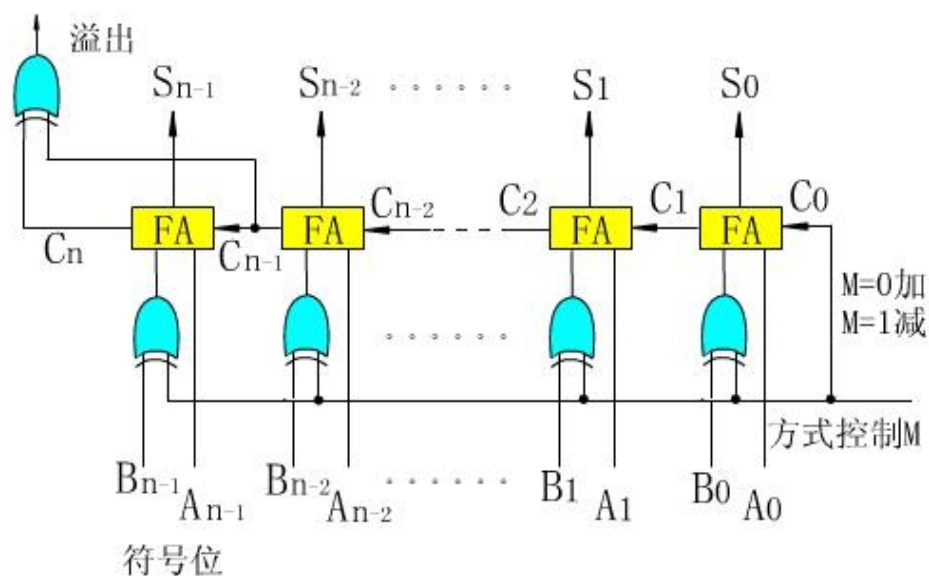


图2-3行波进位的补码加法/加法器

2.3 定点乘法运算

- 1.人工算法与机器算法的同异性
- 2.不带符号的阵列乘法器
- 3.带符号的阵列乘法器

1.人工算法与机器算法的同异性

- $[x]_{\text{原}} = x_f \cdot x_{n-1} \dots x_1 x_0$ $[y]_{\text{原}} = y_f \cdot y_{n-1} \dots y_1 y_0$
- $[x.y]_{\text{原}} = (x_f \oplus y_f) + (x_{n-1} \dots x_1 x_0) \cdot (y_{n-1} \dots y_1 y_0)$
- 用习惯方法求乘积如下：

设 $x = 1101$, $y = 1011$

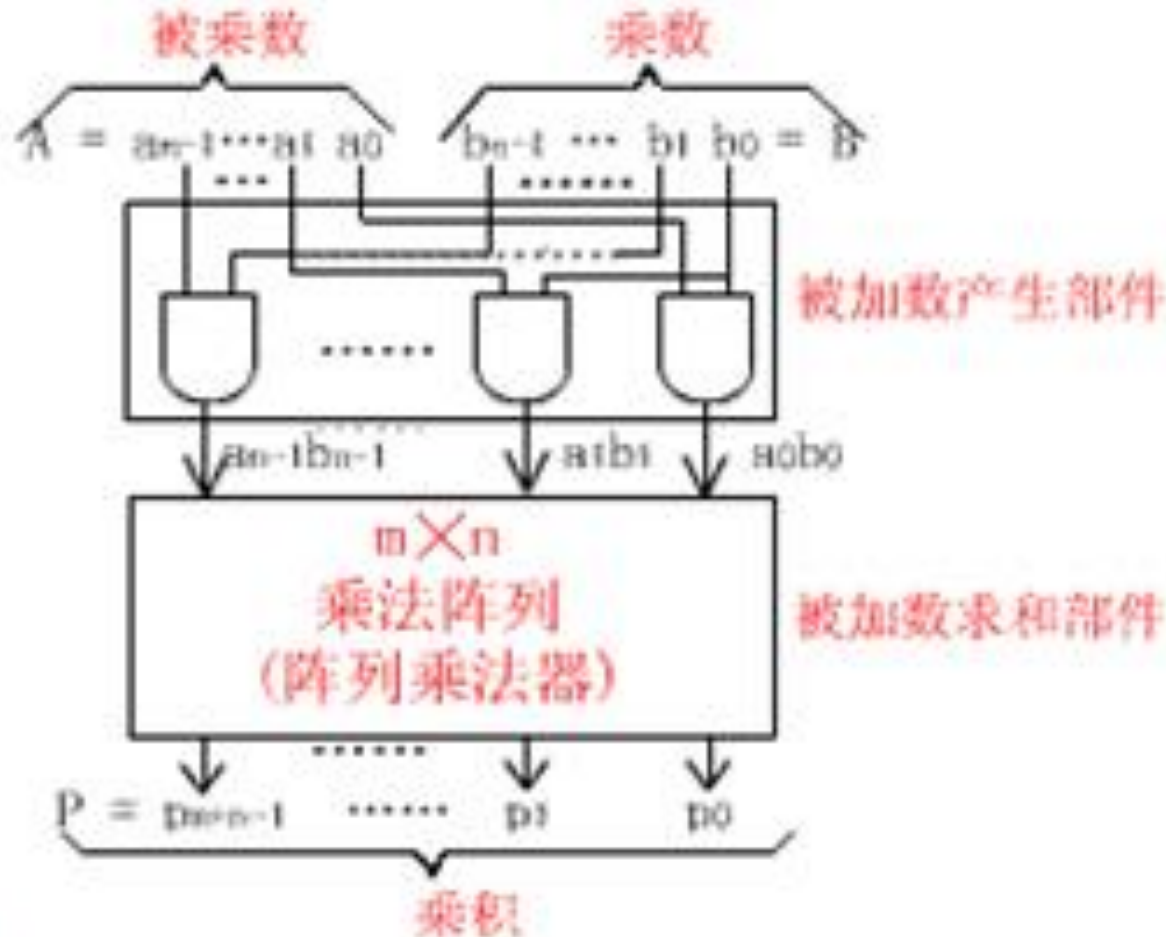
$$\begin{array}{r}
 \begin{array}{cccccc}
 & & & & 1 & 1 & 0 & 1 & (x) \\
 \times & & & & 1 & 0 & 1 & 1 & (y) \\
 \hline
 & & & & 1 & 1 & 0 & 1 \\
 & & 1 & 1 & 0 & 1 & & & \\
 & & & 0 & 0 & 0 & 0 & & \\
 + & & 1 & 1 & 0 & 1 & & & \\
 \hline
 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & (z)
 \end{array}
 \end{array}$$

1.人工算法与机器算法的同异性

- n 位乘 n 位积可能为 $2n$ 位。乘积的最后是所有部分积之和

采用流水式阵列乘法器，取代串行方案。

2.不带符号位的阵列乘法器



不带符号阵列乘法器逻辑图

2.不带符号位的阵列乘法器

延迟时间是 $T + (n-1) \cdot 6T + (n-2) \cdot 2T + 3T$

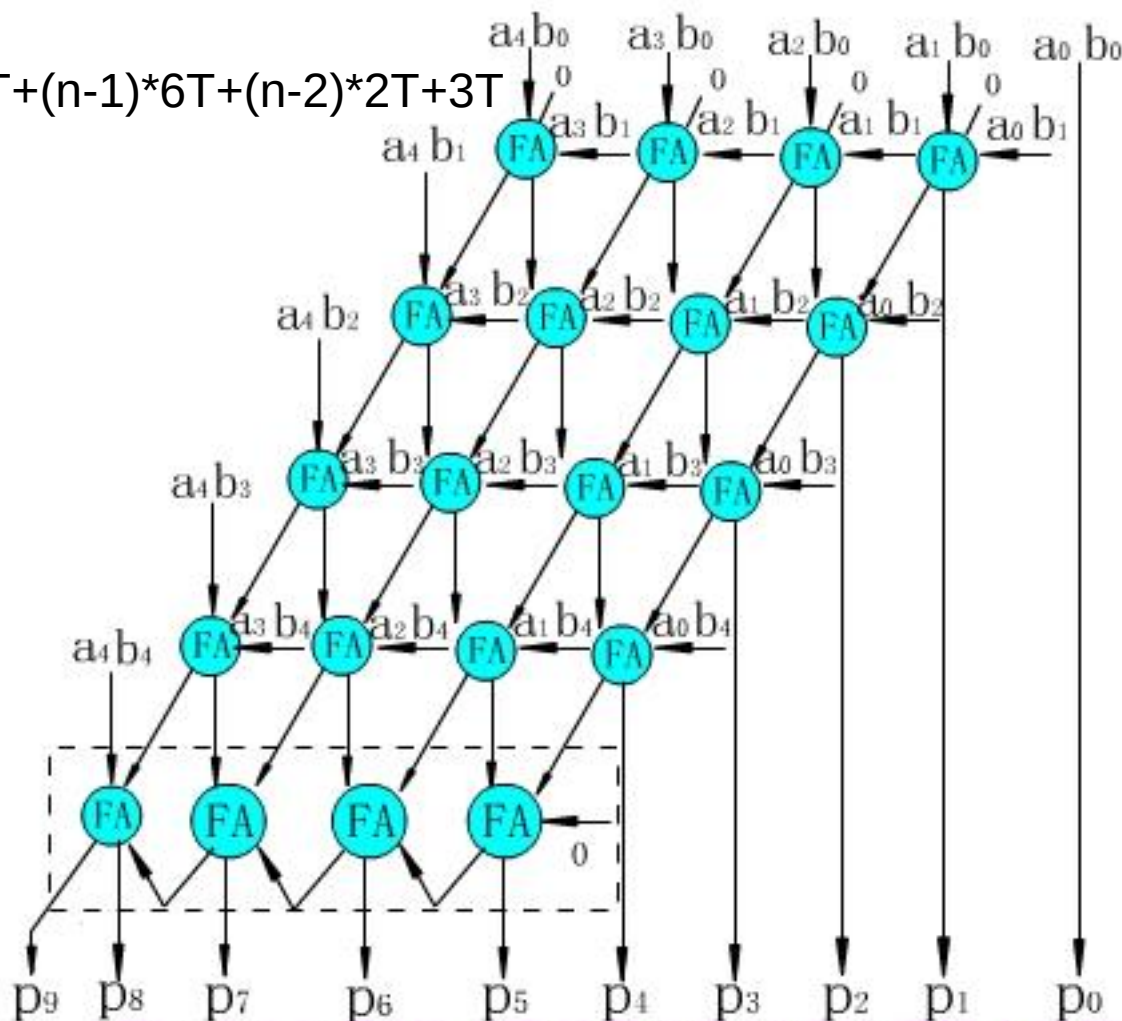


图2.5 5位乘5位不带符号的阵列乘法逻辑电路图

2.不带符号位的阵列乘法器

[例19] 参见图2.5，已知不带符号的二进制整数 $A=11011$ ， $B=10101$ ，求每一部分乘积项 $a_i b_j$ 的值与 $p_9 p_8 \dots p_0$ 的值。

解：

$$\begin{array}{r}
 \times \quad 11011 = A (27_{10}) \\
 \quad 10101 = B (21_{10}) \\
 \hline
 \quad \quad 11011 \\
 \quad 00000 \\
 \quad 11011 \\
 \quad 00000 \\
 + 11011 \\
 \hline
 1000110111 = P
 \end{array}$$

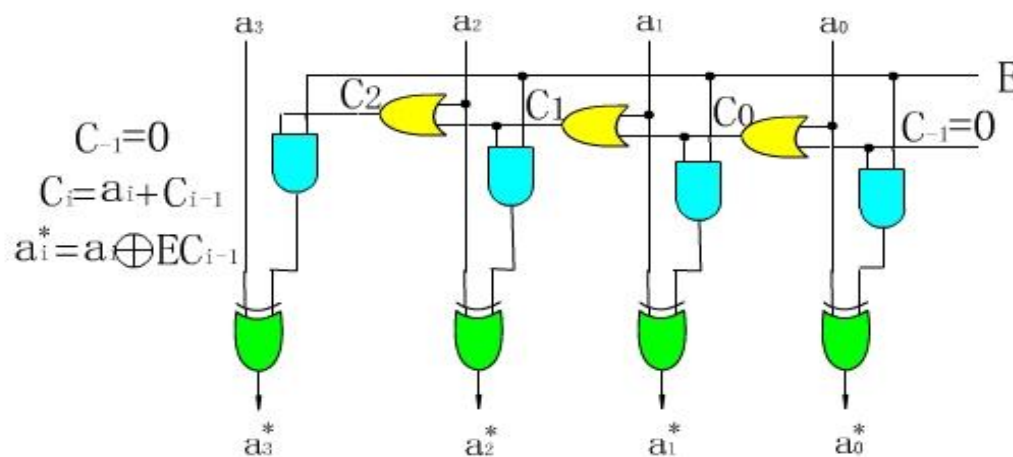
$a_4 b_0=1, a_3 b_0=1, a_2 b_0=0, a_1 b_0=1, a_0 b_0=1$
 $a_4 b_1=0, a_3 b_1=0, a_2 b_1=0, a_1 b_1=0, a_0 b_1=0$
 $a_4 b_2=1, a_3 b_2=1, a_2 b_2=0, a_1 b_2=1, a_0 b_2=1$
 $a_4 b_3=0, a_3 b_3=0, a_2 b_3=0, a_1 b_3=0, a_0 b_3=0$
 $a_4 b_4=1, a_3 b_4=1, a_2 b_4=0, a_1 b_4=1, a_0 b_4=1$

$$P = p_9 p_8 p_7 p_6 p_5 p_4 p_3 p_2 p_1 p_0 = 1000110111 (567_{10})$$

3.带符号位的阵列乘法器

- 求补电路

原理：算前求补—乘法器—算后求补，见下图



延迟时间是 $(n-1)T+4T$

图2.6 对2求补器电路图

3. 带符号的阵列乘法器

- 求补电路小结
- $E=0$ 时，输入和输出相等
- $E=1$ 时，则从数最右端往左边扫描，直到第一个1的时候，该位和右边各位保持不变，左边各数值位按位取反
- 可以用符号作为 E 的输入
- 原：1.11110 补：1.00010

不变，左边数值位取反

3. 带符号的阵列乘法器

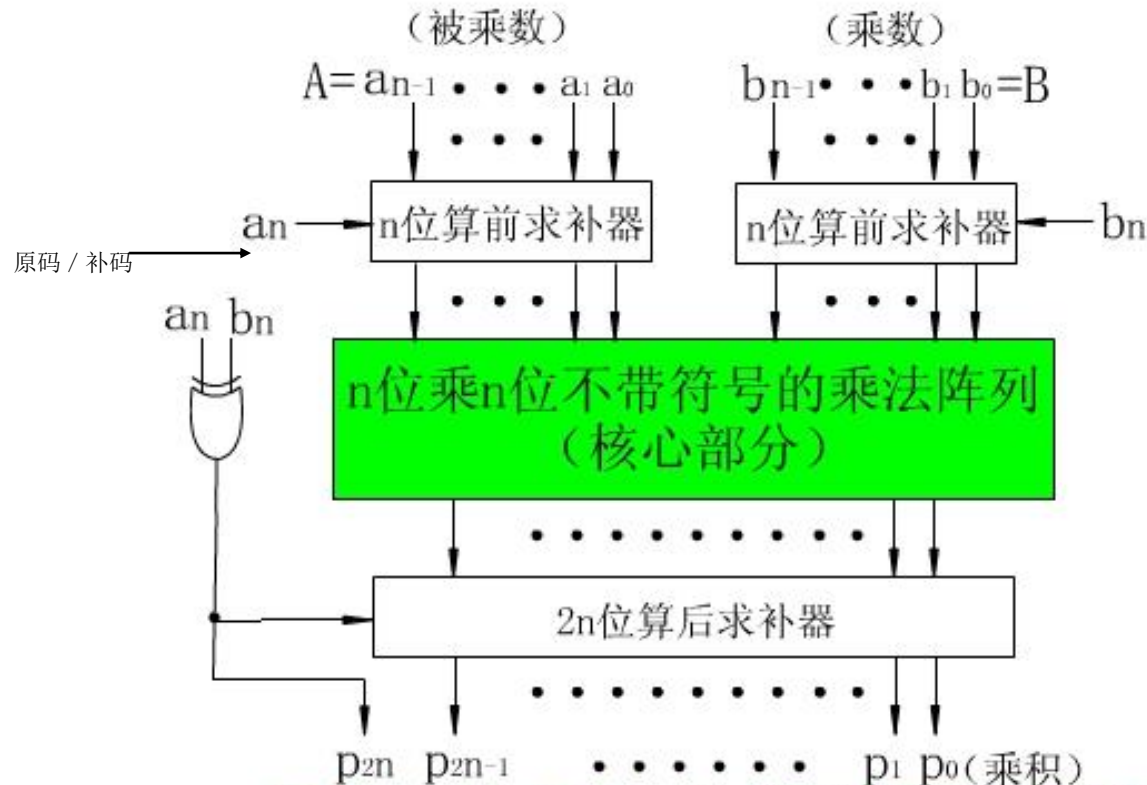


图2.7 $(n+1)$ 位乘 $(n+1)$ 位带补码的阵列乘法器

[例20] 设 $x=+15$, $y=-13$, 用带求补器的原码阵列乘法器求出乘积 $x \cdot y = ?$

解: $[x]_{\text{原}} = 01111$, $[y]_{\text{原}} = 11101$, $|x| = 1111$, $|y| = 1101$

符号位运算: $0 \oplus 1 = 1$

$$\begin{array}{r}
 \times \quad \quad \quad 1111 \\
 \quad \quad \quad 1101 \\
 \hline
 \quad \quad \quad 1111 \\
 \quad \quad 0000 \\
 \quad 1111 \\
 + \quad 1111 \\
 \hline
 11000011
 \end{array}$$

乘积符号为1, 算后求补器输出11000011, $[x \times y]_{\text{原}} = 111000011$
 换算成二进制数真值是 $x \cdot y = (-11000011)_2 = (-195)_{10}$

[例21] 设 $x=-15$, $y=-13$, 用带求补器的补码阵列乘法器求出乘积 $x \cdot y=?$ 并用十进制数乘法进行验证。

解: $[x]_{\text{补}}=10001$, $[y]_{\text{补}}=10011$, 乘积符号位运算: $1 \oplus 1=0$

尾数部分算前求补器输出 $|x|=1111$, $|y|=1101$

$$\begin{array}{r}
 1111 \\
 \times 1101 \\
 \hline
 1111 \\
 0000 \\
 1111 \\
 + 1111 \\
 \hline
 11000011
 \end{array}$$

乘积符号为0, 算后求补器输出11000011, $[x \times y]_{\text{补}}=011000011$

补码二进制数真值 $x \cdot y = 0 \times 2^8 + 1 \times 2^7 + 1 \times 2^6 + 1 \times 2^1 + 1 \times 2^0$
 $= (+195)_{10}$

十进制数乘法验证 $x \cdot y = (-15) \times (-13) = +195$

2.4 定点除法运算

2.4.1 原码除法算法原理

2.4.2 并行除法器

2.4.1 原码除法算法原理

设有n位定点小数（定点整数也适用）

被除数x, $[x]_{\text{原}} = x_f \cdot x_{n-1} \dots x_1 x_0$

除数y, $[y]_{\text{原}} = y_f \cdot y_{n-1} \dots y_1 y_0$

则有商 $q = x/y$,

$$[q]_{\text{原}} = (x_f \oplus y_f) + (0.x_{n-1} \dots x_1 x_0 / 0.y_{n-1} \dots y_1 y_0)$$

商的符号运算 $q_f = x_f \oplus y_f$ 与原码乘法一样，
用模2求和得到。

下面是人工算法例子。

1. 定点原码除法

定点原码一位除法实现方案（手工） $0.10010 / 0.1011$

$$\begin{array}{r}
 0.1011 \overline{) 0.10010} \\
 \underline{-0.01011} \\
 0.001110 \\
 \underline{-0.001011} \\
 0.0000110 \\
 \underline{-0.00001011} \\
 0.000001100 \\
 \underline{-0.000001011} \\
 0.000000001
 \end{array}$$

商 q

$x(r_0)$ 被除数

$2^{-1}y$ 除数右移1位,减除数

r_1 得余数r1

$2^{-2}y$ 除数右移1位,减除数

r_2 得余数r2

$2^{-3}y$ 除数右移1位,不减除数

r_3 得余数r3

$2^{-4}y$ 除数右移1位,减除数

r_4 得余数r4

◆商0还是商1人可以比较后确定，计算机如何确定？

2.不恢复余数的除法

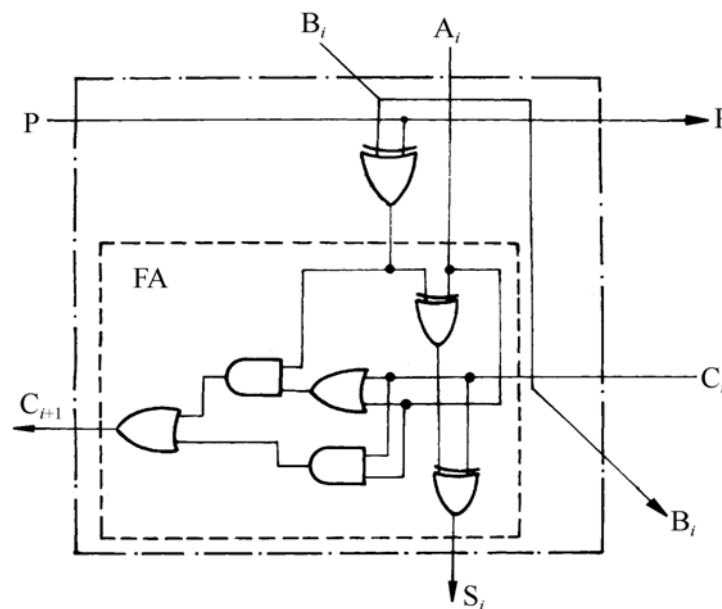
- 人工除法时，人可以比较被除数（余数）和除数的大小来确定商1（够减）或商0（不够减）
- 机器除法时，余数为正表示够减，余数为负表示不够减。不够减时必须恢复原来余数，才能继续向下运算。这种方法叫恢复余数法。
- 不恢复余数法（**加减交替法**）
余数为正，商1，下次除数右移做减法；
余数为负，商0，下次除数右移做加法。
控制简单，有规律。

2.4.2 并行除法器

1. 可控加法/减法(CAS)单元

原理：采用不恢复余数(加减交替)法

- $P=0$ ，作加法运算
- $P=1$ ，作减法运算



(a) 可控加法/减法(CAS)单元的逻辑图

2.4.2 并行除法器

2.不恢复余数的阵列除法器

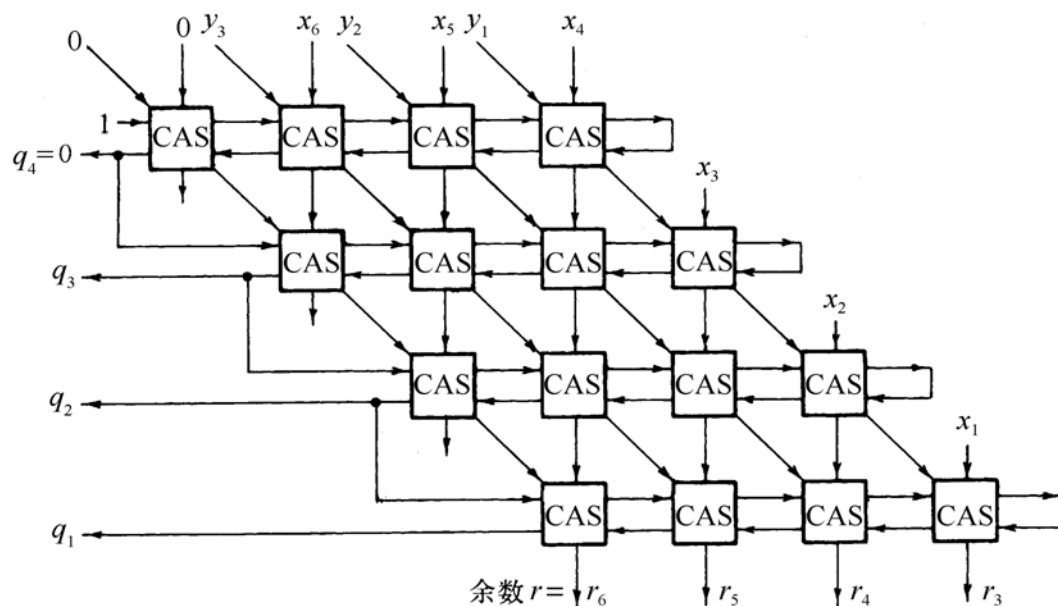
被除数 $x=0.x_6x_5x_4x_3x_2x_1$

除数 $y=0.y_3y_2y_1$

商数 $q=0.q_3q_2q_1$

余数 $r=0.00r_6r_5r_4r_3$

除数右移



(b) 4 位除 4 位阵列除法器



2.4.2 并行除法器

[例23] $x=0.101001$, $y=0.111$, 求 $x \div y$ 。

[解:]

移位运算

1. 概述

移位运算，也称移位操作，是指在小数点位置固定不变的情况下，将一个二进制数左移或右移 n 位。

对于一个定点数，将其左移 n 位，相当于该数乘以 2^n ；

将其右移 n 位，相当于该数除以 2^n 。

思考

计算机总是用定长的二进制位来表示一个数据。

当数据左移或右移 n 位后，必然会使其低 n 位或高 n 位出现空位。

那么，对于空出来的数位应该填补0还是填补1呢？

这与机器数被当作有符号数还是无符号数有关。

①有符号数的移位称为算术移位，

②无符号数的移位称为逻辑移位。



2.2.2 移位运算

2. 算术移位规则

总原则：符号位不变

	码 制	添补代码
正数	原码、补码、反码	0
负数	原 码	0
	补 码	左移 添 0 右移 添 1
	反 码	1

2.4.2 并行除法器

[例23] $x=0.101001$, $y=0.111$, 求 $x \div y$ 。

[解:]

2.4.2 并行除法器

[例23] $x = 0.101001$, $y = 0.111$, 求 $x \div y$ 。

[解:] $[x]_{\text{补}} = 0.101001$, $[y]_{\text{补}} = 0.111$, $[-y]_{\text{补}} = 1.001$

$+ [-y]_{\text{补}}$ $0.1\ 0\ 1\ 0\ 0\ 1$;被除数
 $1.0\ 0\ 1$;第一步减除数y

$+ [y]_{\text{补}} \rightarrow$ $1.1\ 1\ 0\ 0\ 0\ 1$ $<0\ q_4=0$;余数为负,商0
 $0.0\ 1\ 1\ 1$;除数右移1位加

$+ [-y]_{\text{补}} \rightarrow$ $0.0\ 0\ 1\ 1\ 0\ 1$ $>0\ q_3=1$;余数为正,商1
 $1.1\ 1\ 0\ 0\ 1$;除数右移2位减

$+ [y]_{\text{补}} \rightarrow$ $1.1\ 1\ 1\ 1\ 1\ 1$ $<0\ q_2=0$;余数为负,商0
 $0.0\ 0\ 0\ 1\ 1\ 1$;除数右移3位加

$0.0\ 0\ 0\ 1\ 1\ 0$ $>0\ q_1=1$;余数为正,商1

商 $q = q_4.q_3q_2q_1 = 0.101$, 余数 $r = (0.00r_6r_5r_4r_3) = 0.000110$

- 加减交替阵列除法器中，需要另外增加一个异或门来求商的符号
- 被除数 x ， $x = x_f \cdot x_{n-1} \dots x_1 x_0$
除数 y ， $y = y_f \cdot y_{n-1} \dots y_1 y_0$
商的符号位需要单独考虑
 - $[|x|]_{\text{补}}$ 、 $[|y|]_{\text{补}}$ 、 $[-|y|]_{\text{补}}$ ，
 - 先用被除数减去除数($|x| - |y| = |x| + (-|y|) = |x| + [-|y|]_{\text{补}}$)，
当余数为正时，商上1，除数右移一位，再减去除数；
当余数为负时，商上0，除数右移一位，再加上除数
 - 商的符号运算 $q_f = x_f \oplus y_f$
 - 当第 $n+1$ 步余数为负时，需加上 $|Y|$ 得到第 $n+1$ 步正确的余数(余数与被除数同号)。

2.5 定点运算器的组成

2.5.1 逻辑运算

2.5.2 多功能算术/逻辑运算单元ALU

2.5.3 内部总线

2.5.4 定点运算器的基本结构



2.5.1 逻辑运算

- 1.逻辑非运算
- 2.逻辑加运算
- 3.逻辑乘运算
- 4.逻辑异运算

1.逻辑非运算

$X = X_0X_1X_2\dots X_n$ ，对 x 求逻辑非，则有

$$\overline{X} = Z = Z_0Z_1Z_2\dots Z_n$$

$$z_i = \overline{x_i}, i=0,1,2,\dots,n$$

1.逻辑非运算

[例24] $x_1=01001001$, $x_2=11110000$, 求 $\overline{x_1}$, $\overline{x_2}$
解:

$$\overline{x_1} = 10110100$$

$$\overline{x_2} = 00001111$$

2.逻辑加运算

$$X = x_0x_1x_2\cdots x_n, y = y_0y_1y_2\cdots y_n$$

则有

$$x+y = z = z_0z_1z_2\cdots z_n$$

$$z_i = x_i+y_i, i=0,1,2,\dots,n$$

2.逻辑加运算

[例25] $x=10100001$, $y=100111011$, 求 $x+y$
解:

$$\begin{array}{r} 10100001 \quad x \\ + 100111011 \quad y \\ \hline 101111011 \quad z \end{array}$$

即 $x+y = 101111011$

3.逻辑乘运算

$$X = x_0x_1x_2\cdots x_n, y = y_0y_1y_2\cdots y_n$$

则有

$$x \cdot y = z = z_0z_1z_2\cdots z_n$$

$$z_i = x_i \cdot y_i, i=0,1,2,\dots,n$$

3.逻辑乘运算

[例26] $x=10111001$, $y=11110011$, 求 $x \cdot y$
解:

$$\begin{array}{r} 10111001 \quad x \\ \cdot 11110011 \quad y \\ \hline 10110001 \quad z \end{array}$$

即 $x \cdot y = 10110001$

4.逻辑异（异或）运算

$$X = x_0x_1x_2\cdots x_n, y = y_0y_1y_2\cdots y_n$$

则有

$$x \oplus y = z = z_0z_1z_2\cdots z_n$$

$$z_i = x_i \oplus y_i, i=0,1,2,\dots,n$$

4.逻辑异运算

[例27] $x=10101011$, $y=11001100$, 求 $x+y$
解:

$$\begin{array}{r} 10101011 \quad x \\ \oplus 11001100 \quad y \\ \hline 01100111 \quad z \end{array}$$

即 $x \oplus y = 01100111$

2.5.2 多功能算术/逻辑运算单元ALU

1. 先行进位原理
2. ALU基本思想
3. 逻辑表达式
3. 算术逻辑运算的实现
4. 两级先行进位的ALU

1. 先行进位基本思想

一位全加器(FA)的逻辑表达式为

$$S_i = A_i \oplus B_i \oplus C_i$$

$$\begin{aligned} C_{i+1} &= A_i B_i + B_i C_i + C_i A_i \\ &= A_i B_i + (A_i \oplus B_i) C_i \end{aligned}$$

设： $G_i = A_i B_i$ $P_i = A_i \oplus B_i$

则有： $S_i = P_i \oplus C_i$

$$C_{i+1} = G_i + P_i C_i$$

G_i ：本次进位产生

P_i ：本次进位传递

1. 先行进位基本思想

4位行波进位加法器的逻辑表达式为:

$$C_0$$

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 C_1$$

$$C_3 = G_2 + P_2 C_2$$

$$C_4 = G_3 + P_3 C_3$$

行波（串行）进位中，每一级全加器的进位延时为 $2T$ 。在字长为 n 位时，若不考虑 G_i 和 P_i 的形成时间，从 $C_0 \rightarrow C_n$ 的最长延时为 $2nT$ 。

1. 先行进位基本思想

4位先行进位的逻辑表达式为:

$$C_0$$

$$C_1 = G_0 + P_0 C_0$$

$$\begin{aligned} C_2 &= G_1 + P_1 C_1 = G_1 + P_1 (G_0 + P_0 C_0) \\ &= G_1 + G_0 P_1 + P_0 P_1 C_0 \end{aligned}$$

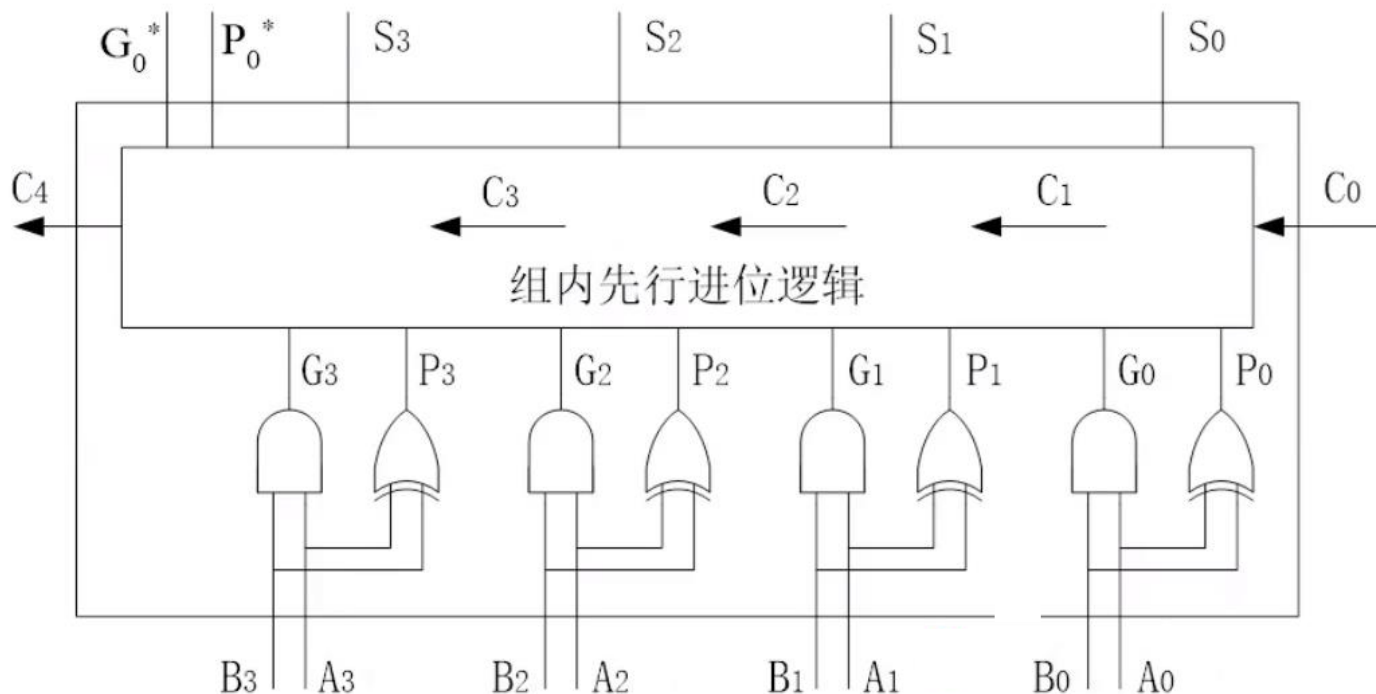
$$\begin{aligned} C_3 &= G_2 + P_2 C_2 = G_2 + P_2 (G_1 + G_0 P_1 + P_0 P_1 C_0) \\ &= G_2 + G_1 P_2 + G_0 P_1 P_2 + P_0 P_1 P_2 C_0 \end{aligned}$$

$$\begin{aligned} C_4 &= G_3 + P_3 C_3 = G_3 + P_3 (G_2 + \dots + P_0 P_1 P_2 C_0) \\ &= G_3 + G_2 P_3 + G_1 P_2 P_3 + G_0 P_1 P_2 P_3 + P_0 P_1 P_2 P_3 C_0 \end{aligned}$$

先行（并行）进位中，所有输出仅由 G_i 和 P_i 及最低进位输入 C_0 决定，因此各级进位输出可以同时产生

1. 先行进位基本思想

先行进位CLA (Carry Look Ahead) 示意图:

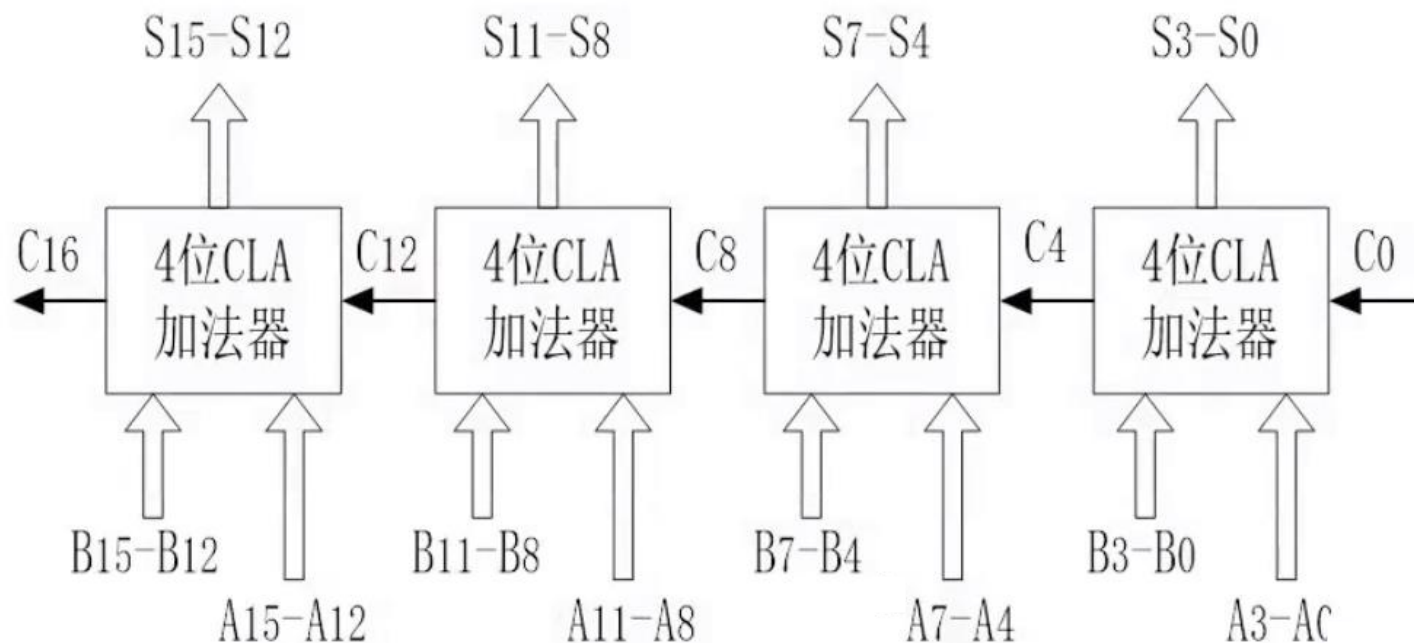


先行进位中, 若不考虑 G_i 和 P_i 的形成时间, 从 $C_0 \rightarrow C_n$ 的最长延时为 $2T$, 而与字长 n 无关。考虑电路复杂性, 实际设计通常以4位为一组。

分组先行进位

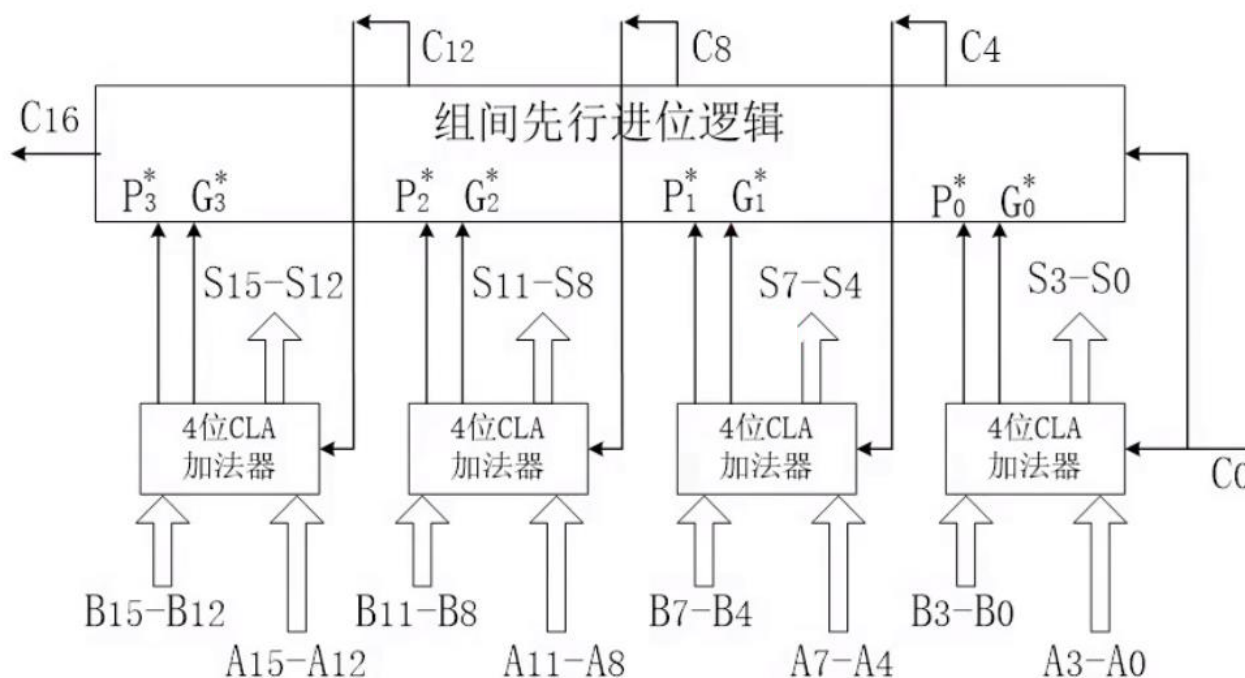
(1)组内并行，组间串行：

以16位加法器为例，可分4组，每组4位。每组组内的延迟时间是 $2T$ 。该加法器总延时为： $8T$ 。



分组先行进位

(2)组内并行，组间并行：

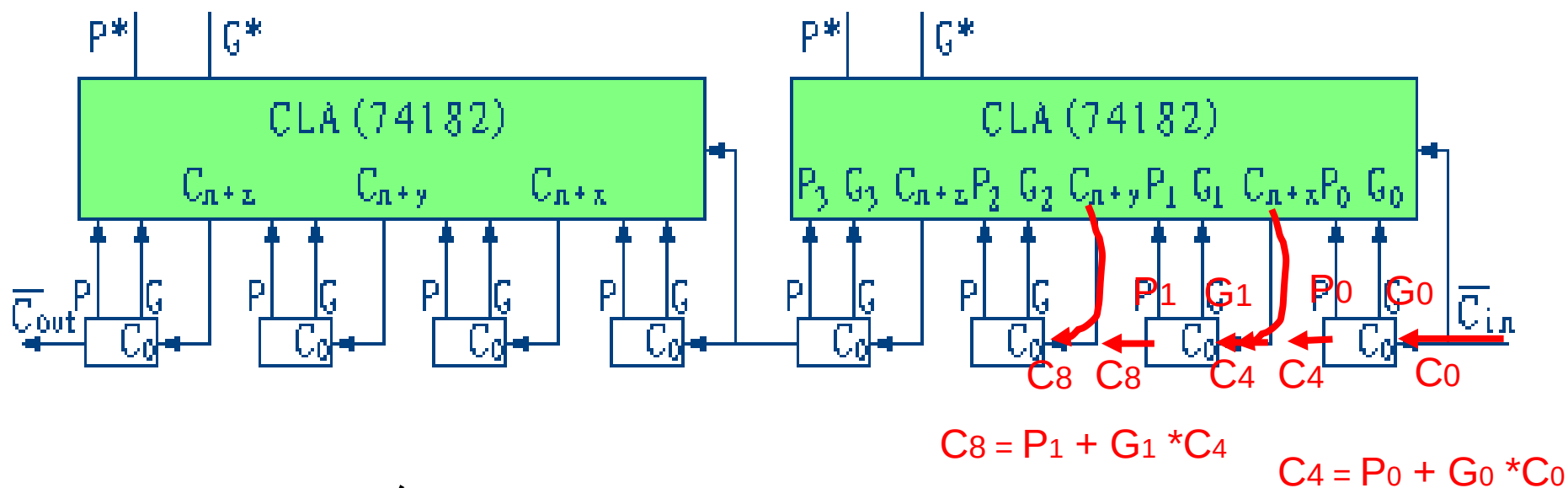


若不考虑 G_i 和 P_i 的形成时间，经过 $2T$ 产生各组的 G_i^* 和 P_i^* ；再经过 $2T$ ，由组间先行进位电路产生 C_4 、 C_8 、 C_{12} 、 C_{16} 。

2. ALU基本思想

- 两级先行进位的ALU
 - 4片（组）的先行进位逻辑
 - $C_{n+x} = G_0 + P_0 C_n$
 - $C_{n+y} = G_1 + P_1 C_{n+x} = G_1 + G_0 P_1 + P_0 P_1 C_n$
 - $C_{n+z} = G_2 + P_2 C_{n+y}$
 - $= G_2 + G_1 P_2 + G_0 P_1 P_2 + P_0 P_1 P_2 C_n$
 - $C_{n+4} = G_3 + P_3 C_{n+z}$
 - $= G_3 + G_2 P_3 + G_1 P_2 P_3 + G_0 P_1 P_2 P_3 + P_0 P_1 P_2 P_3 C_n$
 - $= G^* + P^* C_n$
 - G^* 为成组先行进位发生输出
 - P^* 为成组先行进位传送输出

2. ALU基本思想



2个74182

8个4位74181

图2.13 用两个16位全先行进位逻辑级联组成的32位ALU

2. ALU基本思想

一位全加器(FA)的逻辑表达式为

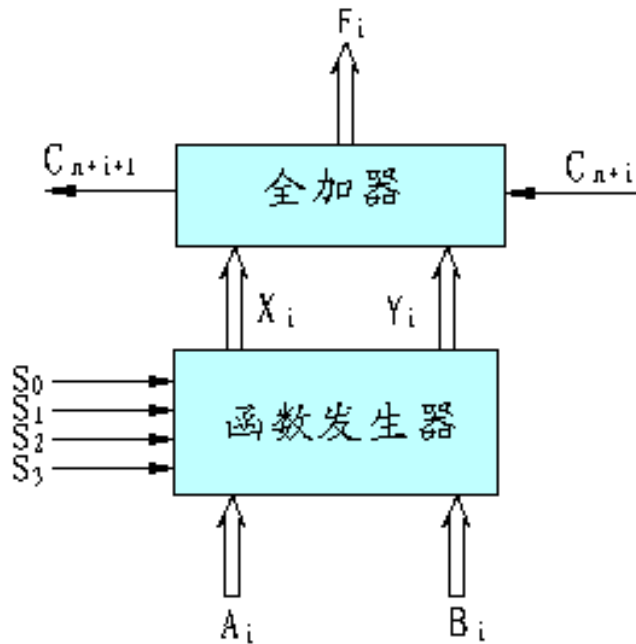
$$F_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_i B_i + B_i C_i + C_i A_i$$

我们将 A_i 和 B_i 先组合成由控制参数 S_0, S_1, S_2, S_3 控制的组合函数 X_i 和 Y_i ,然后再将 X_i, Y_i 和下一位进位数通过全加器进行全加。这样,不同的控制参数可以得到不同的组合函数,因而能够实现多种算术运算和逻辑运算。

2. ALU基本思想

ALU的逻辑图与逻辑表达式



$$F_i = X_i \oplus Y_i \oplus C_{n+i}$$

$$C_{n+i+1} = X_i Y_i + Y_i C_{n+i} + C_{n+i} X_i$$

4位一片， $i=0,1,2,3$

一位ALU逻辑图

2. ALU基本思想

2.逻辑表达式

$X_i Y_i$ 与控制参数和输入量的关系构造如下真值表

S_0 S_1	Y_i	S_2 S_3	X_i
0 0	$\overline{A_i}$	0 0	$\overline{A_i}$
0 1	$\overline{A_i} B_i$	0 1	$\overline{A_i} + B_i$
1 0	$\overline{A_i} \overline{B_i}$	1 0	$\overline{A_i}$
1 1	0	1 1	$\overline{A_i} + \overline{B_i}$

$$Y_i = \overline{S_0} \overline{S_1} \overline{A_i} + \overline{S_0} S_1 \overline{A_i} B_i + S_0 \overline{S_1} \overline{A_i} \overline{B_i}$$

$$X_i = \overline{S_2} \overline{S_3} + \overline{S_2} S_3 (\overline{A_i} + \overline{B_i}) + \overline{S_2} S_3 (\overline{A_i} + B_i) + S_2 S_3 \overline{A_i}$$

2. ALU基本思想

- ALU的某一位逻辑表达式见下:

$$Y_i = \overline{S_0} \overline{S_1} \overline{A_i} + \overline{S_0} S_1 \overline{A_i} B_i + S_0 \overline{S_1} \overline{A_i} \overline{B_i}$$

$$X_i = \overline{S_2} \overline{S_3} + \overline{S_2} S_3 (\overline{A_i} + \overline{B_i}) + S_2 \overline{S_3} (\overline{A_i} + B_i) + S_2 S_3 \overline{A_i}$$



$$X_i = \overline{S_3 A_i B_i + S_2 A_i \overline{B_i}}$$

$$Y_i = \overline{A_i + S_0 B_i + S_1 \overline{B_i}}$$

真值表推出书上X、Y表达式

S_0	S_1	AB	Y	$A + S_0B + S_1\bar{B}$	S_2	S_3	AB	X	$S_3AB + S_2A\bar{B}$
0	0	00	1	1	0	0	00	1	1
0	0	01	1	1	0	0	01	1	1
0	0	10	0	0	0	0	10	1	1
0	0	11	0	0	0	0	11	1	1
0	1	00	0	0	0	1	00	1	1
0	1	01	1	1	0	1	01	1	1
0	1	10	0	0	0	1	10	1	1
0	1	11	0	0	0	1	11	0	0
1	0	00	1	1	1	0	00	1	1
1	0	01	0	0	1	0	01	1	1
1	0	10	0	0	1	0	10	0	0
1	0	11	0	0	1	0	11	1	1
1	1	00	0	0	1	1	00	1	1
1	1	01	0	0	1	1	01	1	1
1	1	10	0	0	1	1	10	0	0
1	1	11	0	0	1	1	11	0	0

由真值表可得: $X = \overline{S_3AB + S_2A\bar{B}}$
 $Y = \overline{A + S_0B + S_1\bar{B}}$

2. ALU基本思想

- ALU的某一位逻辑表达式见下:

$$X_i = \overline{S_3 A_i B_i + S_2 A_i \bar{B}_i}$$

$$Y_i = \overline{A_i + S_0 B_i + S_1 \bar{B}_i}$$

$$F_i = Y_i \oplus X_i \oplus C_{n+i}$$

$$C_{n+i+1} = Y_i + X_i C_{n+i}$$



$$C_{n+i+1} = X_i Y_i + Y_i C_{n+1} + C_{n+1} X_i$$

真值表推出书上X、Y表达式

S_0	S_1	AB	Y	$A + S_0B + S_1\bar{B}$	S_2	S_3	AB	X	$S_3AB + S_2A\bar{B}$
0	0	00	1	1	0	0	00	1	1
0	0	01	1	1	0	0	01	1	1
0	0	10	0	0	0	0	10	1	1
0	0	11	0	0	0	0	11	1	1
0	1	00	0	0	0	1	00	1	1
0	1	01	1	1	0	1	01	1	1
0	1	10	0	0	0	1	10	1	1
0	1	11	0	0	0	1	11	0	0
1	0	00	1	1	1	0	00	1	1
1	0	01	0	0	1	0	01	1	1
1	0	10	0	0	1	0	10	0	0
1	0	11	0	0	1	0	11	1	1
1	1	00	0	0	1	1	00	1	1
1	1	01	0	0	1	1	01	1	1
1	1	10	0	0	1	1	10	0	0
1	1	11	0	0	1	1	11	0	0

由真值表可得: $X = \overline{S_3AB + S_2A\bar{B}}$
 $Y = \overline{A + S_0B + S_1\bar{B}}$

$$Y = XY$$

$$X = X+Y$$

2. ALU基本思想

算术逻辑运算的实现

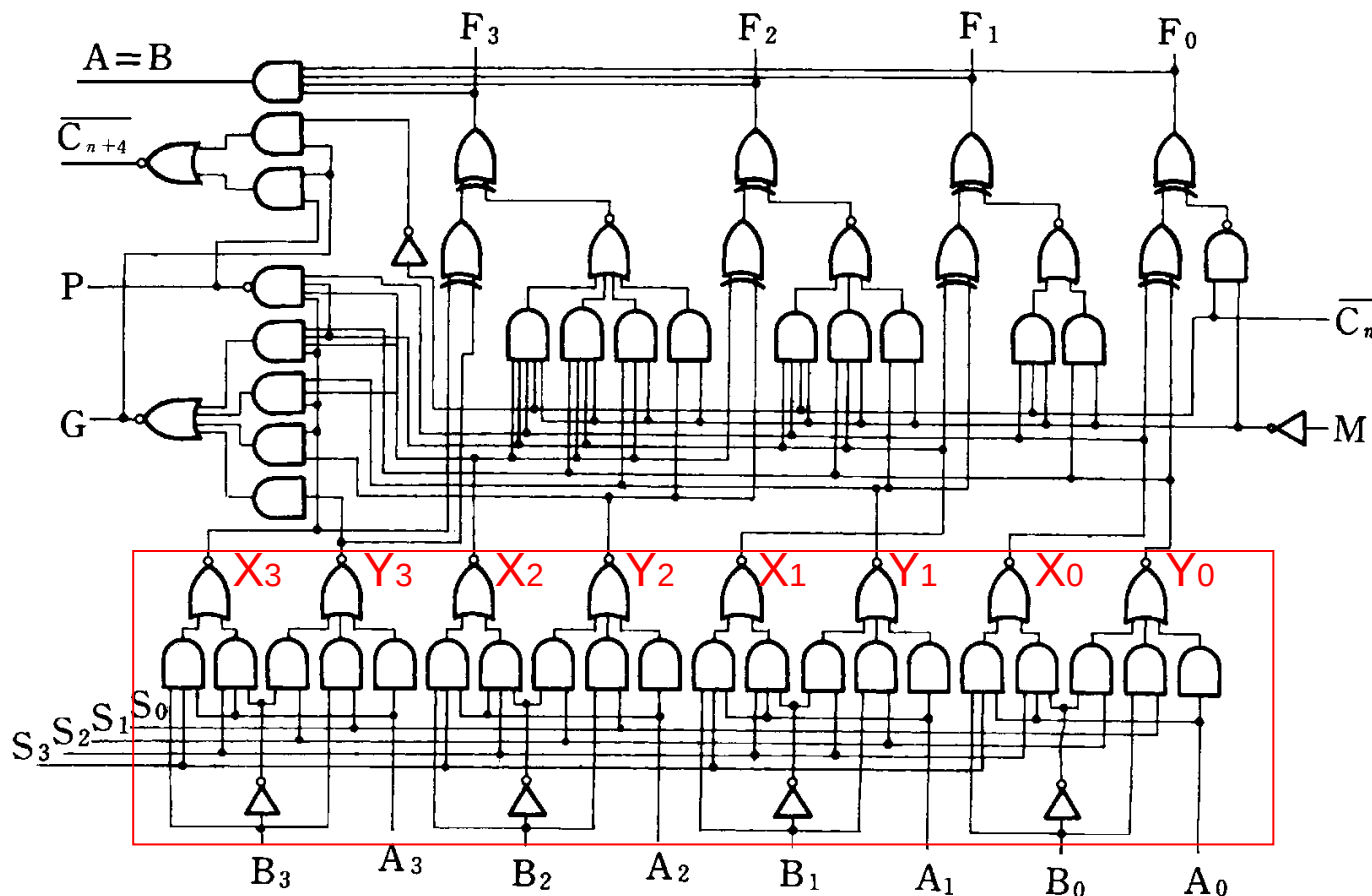


图2.11 正逻辑操作数表示的74181ALU逻辑电路图

2. ALU基本思想

算术逻辑运算的实现

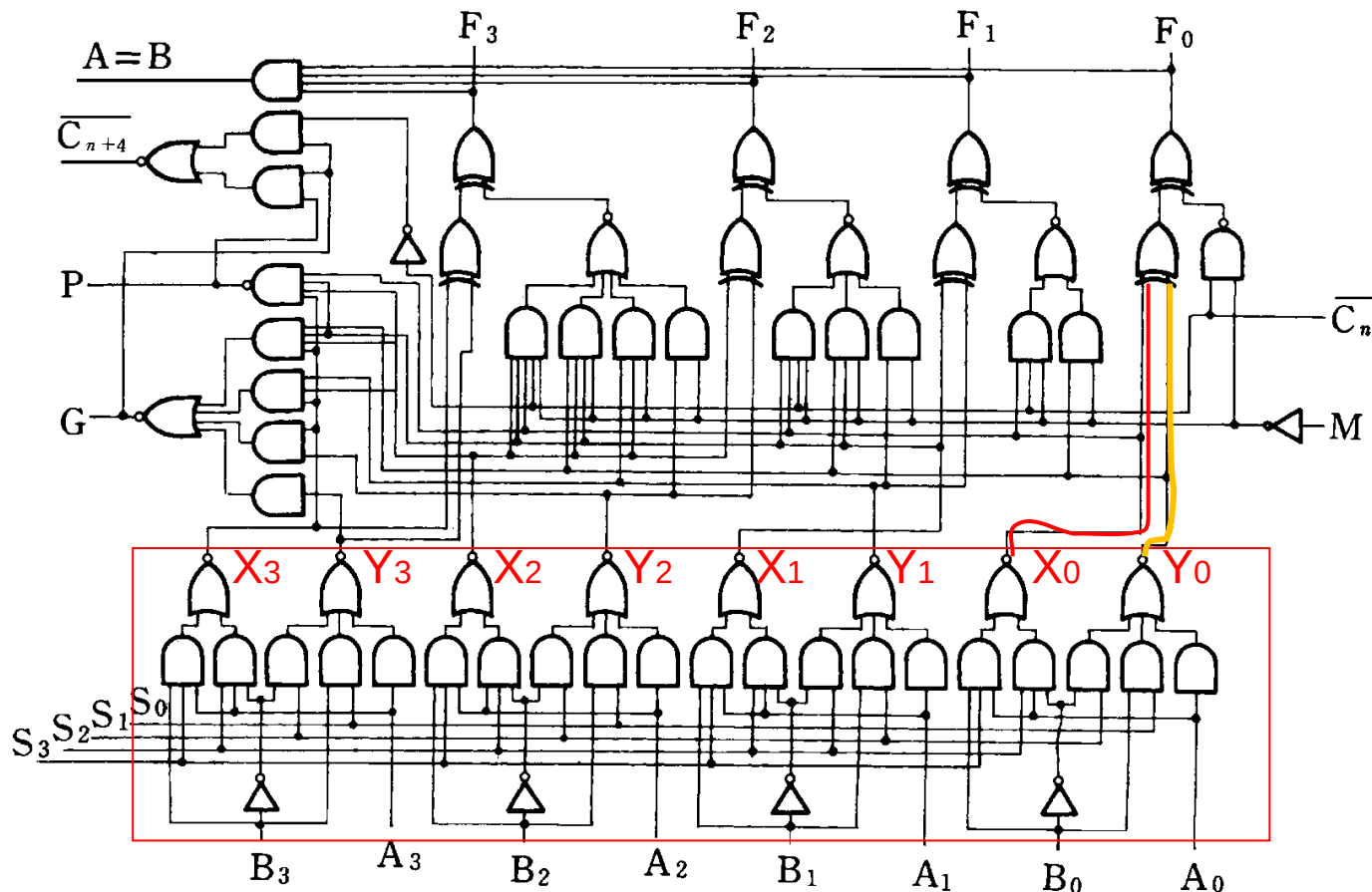
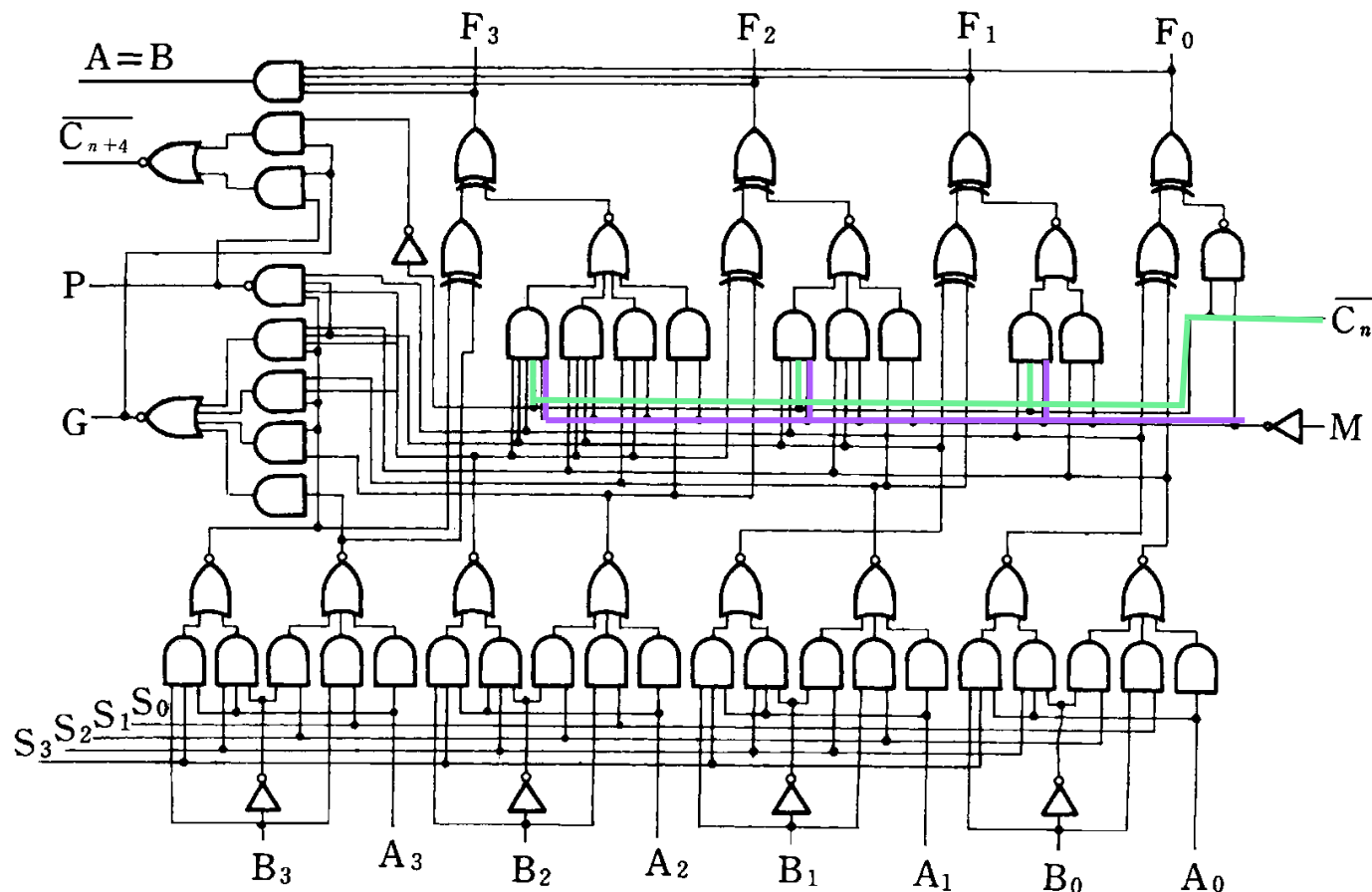


图2.11 正逻辑操作数表示的74181ALU逻辑电路图

2. ALU基本思想



$M=1$ 时，
进位门被
封锁，做
逻辑运算

$M=0$ 时，对
进位信号没
有影响，做
算术运算

图2.11 正逻辑操作数表示的74181ALU逻辑电路图

2. ALU基本思想

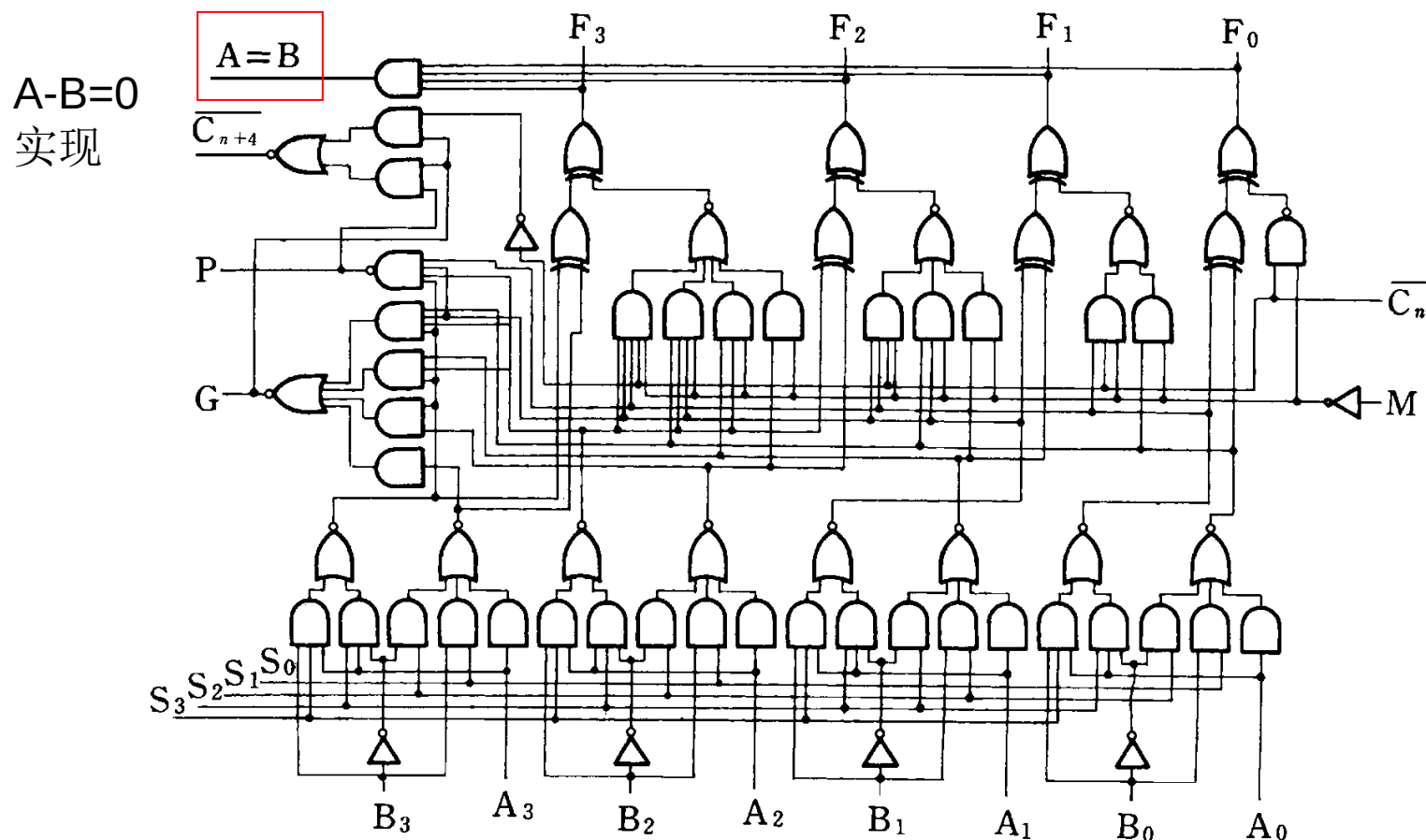


图2.11 正逻辑操作数表示的74181ALU逻辑电路图

2. ALU基本思想

- 算术逻辑运算的实现（74181）
 - $M=0$ 时，对进位信号没有影响，做算术运算
 - $M=1$ 时，进位门被封锁，做逻辑运算
- $A=B$ 端可以判断两个数是否相等。

正负逻辑

- 两种工作方式。对正逻辑操作数来说，算术运算称高电平操作，逻辑运算称正逻辑操作(即高电平为“1”，低电平为“0”)。对于负逻辑操作数来说，正好相反。
- 都有16种算术运算功能和16种逻辑运算功能
- 正负逻辑功能等效性

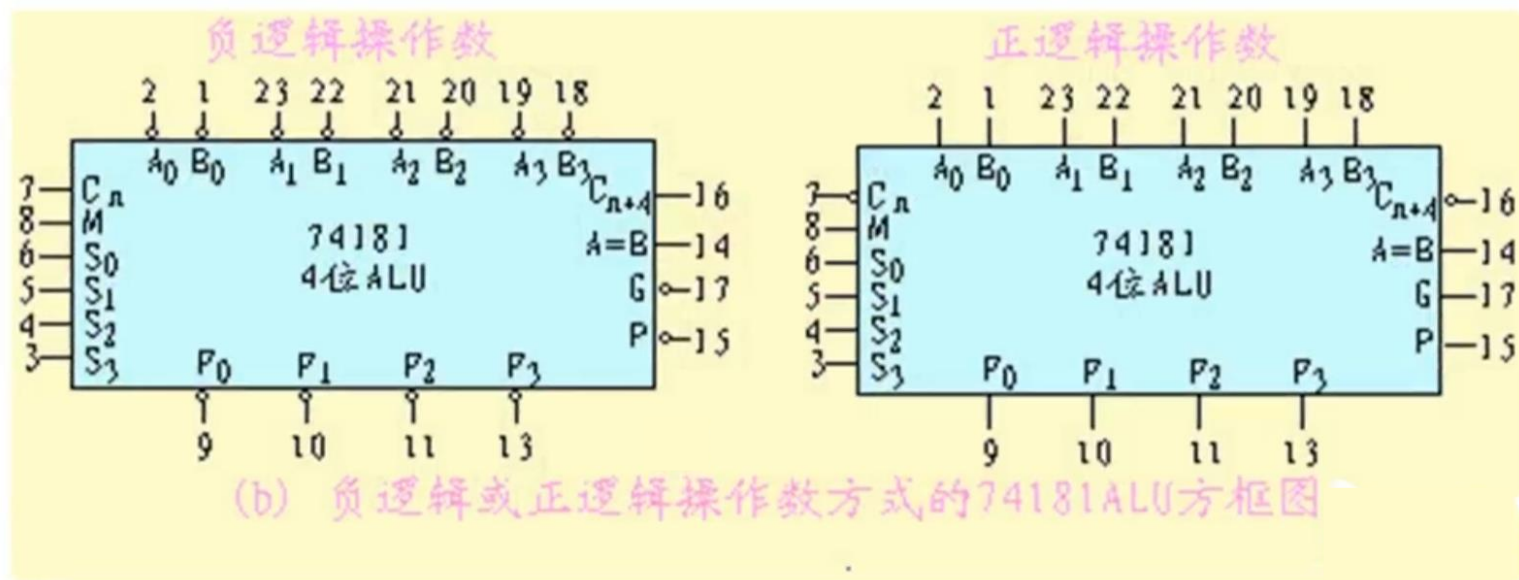


表 2.4 74181ALU 算术/逻辑运算功能表

工作方式选择输入				正逻辑输入与输出	
S_3	S_2	S_1	S_0	逻辑运算 $M=1$	算术运算 $M=0$ $C_n=1$
0	0	0	0	$\bar{A}$	A
0	0	0	1	$\overline{A+B}$	$A+B$
0	0	1	0	$\bar{A}B$	$A+\bar{B}$
0	0	1	1	逻辑 0	减 1
0	1	0	0	$\overline{AB}$	A 加 $A\bar{B}$
0	1	0	1	$\bar{B}$	(A+B) 加 $A\bar{B}$
0	1	1	0	$A\oplus B$	A 减 B 减 1
0	1	1	1	$A\bar{B}$	$A\bar{B}$ 减 1
1	0	0	0	$\bar{A}+B$	A 加 AB
1	0	0	1	$\overline{A\oplus B}$	A 加 B
1	0	1	0	B	(A+B) 加 AB
1	0	1	1	AB	AB 减 1
1	1	0	0	逻辑 1	A 加 A
1	1	0	1	$A+\bar{B}$	(A+B) 加 A
1	1	1	0	$A+B$	($A+\bar{B}$) 加 A
1	1	1	1	A	A 减 1

2.5.3 内部总线

- 内部总线

- 机器内部各部份数据传送频繁,可以把寄存器间的数据传送通路加以归并,组成总线结构。
- 分类
 - 所处位置
 - 内部总线 (CPU内)
 - 外部总线 (系统总线)
 - 逻辑结构
 - 单向传送总线: 信息只能往一个方向传送
 - 双向传送总线: 可以分两个方向 (发送和接收数据)

2.5.3 内部总线

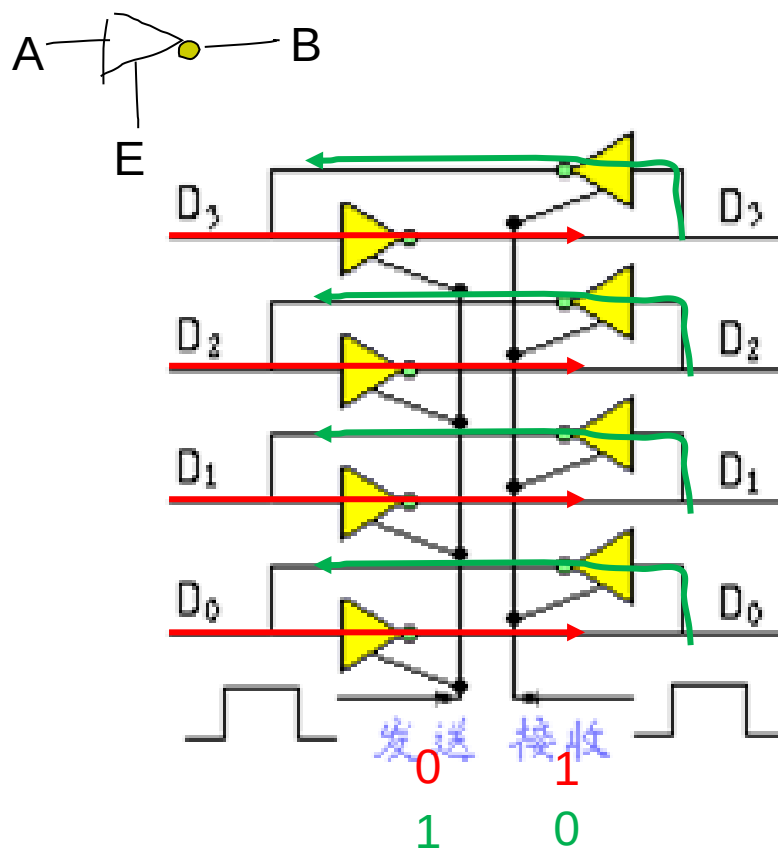


图2.14 由三态门组成的双向数据总线

2.5.4 定点运算器的基本结构

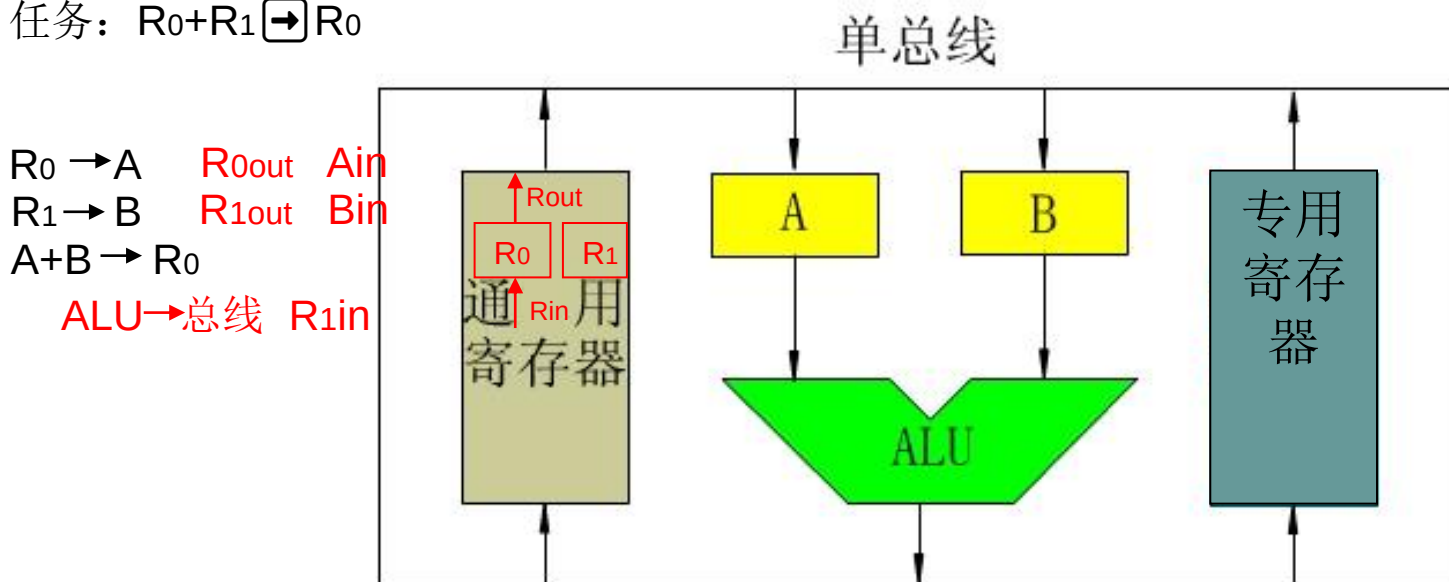
- 运算器包括**ALU**、**阵列乘除器**、**寄存器**、**多路开关**、**三态缓冲器**、**数据总线**等逻辑部件。
- 运算器的设计，主要是围绕着**ALU**和寄存器同数据总线之间如何传送操作数和运算结果而进行的。

2.5.4 定点运算器的基本结构

1. 单总线结构的运算器

由于所有部件都接到同一总线上，所以数据可以在任何两个寄存器之间，或者在任一个寄存器和ALU之间传送

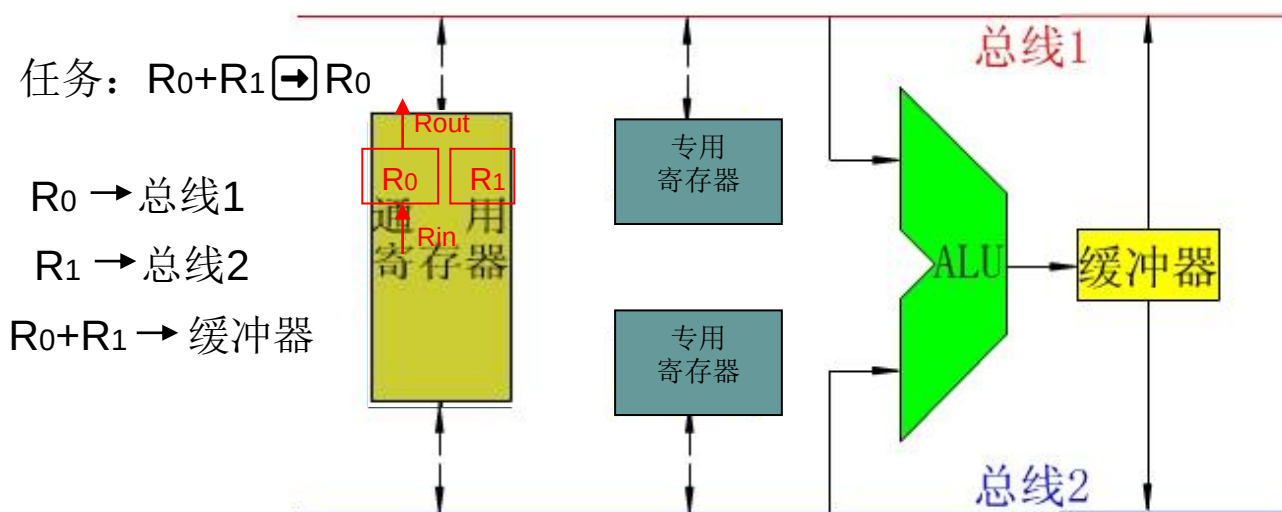
任务： $R_0 + R_1 \rightarrow R_0$



(a) 单总线结构的运算器

2.5.4 定点运算器的基本结构

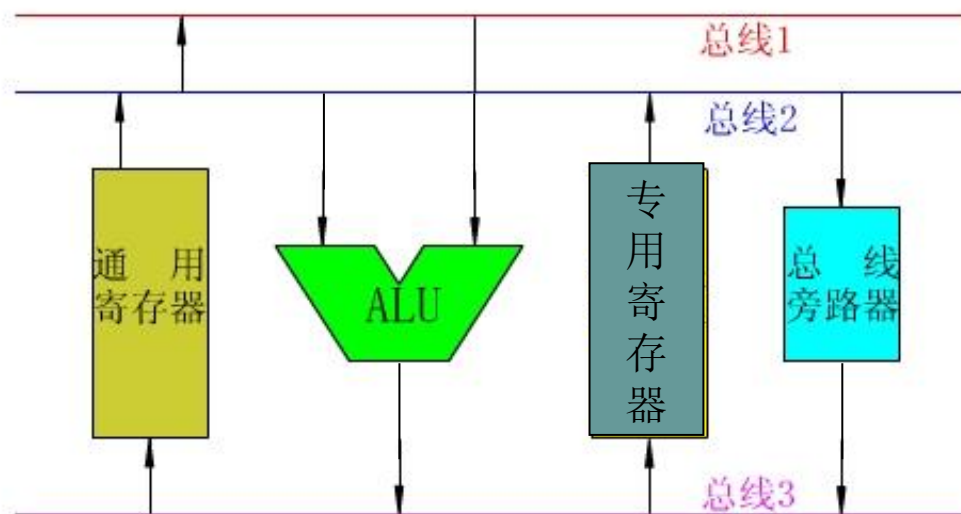
2. 双总线结构的运算器



(b) 双总线结构的运算器

2.5.4 定点运算器的基本结构

3.三总线结构的运算器



(c) 三总线结构的运算器

2.6 浮点运算方法和浮点运算器

2.6.1 浮点加法、减法运算

2.6.2 浮点乘法、除法运算

2.6.3 浮点运算流水线

2.6.4 浮点运算器实例

2.6.1 浮点加法、减法运算

1.浮点加减运算

设有两个浮点数 x 和 y , 它们分别为

$$x = 2^{E_x} \cdot M_x$$

$$y = 2^{E_y} \cdot M_y$$

其中 E_x 和 E_y 分别为数 x 和 y 的阶码, M_x 和 M_y 为数 x 和 y 的尾数。两浮点数进行加法和减法的运算规则是

$$x \pm y = (M_x 2^{E_x - E_y} \pm M_y) 2^{E_y},$$

$$\text{设 } E_x \leq E_y$$

2.6.1 浮点加法、减法运算

2. 浮点运算步骤如下：

- (1) 0 操作数检查；
- (2) 比较阶码并完成对阶（小阶向大阶对齐，**变形补码**）；
- (3) 尾数加或减运算（**变形补码**）；
- (4) 结果规格化；
- (5) 舍入处理。

2.6.1 浮点加法、减法运算

1、对阶

已知 $x = 0.1101 \times 2^{01}$ $y = (-0.1010) \times 2^{11}$ 求 $x+y$

解: $[x]_{\text{补}} = 00,01; 00.1101$ $[y]_{\text{补}} = 00,11; 11.0110$

(1) 对阶

① 求阶差

$$\begin{array}{r} [\Delta E]_{\text{补}} = [E_x]_{\text{补}} - [E_y]_{\text{补}} = 00,01 \\ + \quad 11,01 \\ \hline 11,10 \end{array}$$

阶差为负 (-2) $\therefore M_x \rightarrow 2 \quad E_x + 2$

② 对阶

$[x]_{\text{补}}' = 0011; 000011$

2、尾数求和

$$\begin{array}{r} [M_x]_{\text{补}}' = 00.0011 \quad \text{对阶后的}[M_x]_{\text{补}}' \\ + \quad [M_y]_{\text{补}} = 11.0110 \\ \hline 11.1001 \end{array}$$

$\therefore [x+y]_{\text{补}} = 00,11; 11.1001$

2.6.1 浮点加法、减法运算

2. 浮点运算步骤如下：

- (1) 0 操作数检查；
- (2) 比较阶码并完成对阶（小阶向大阶对齐，变形补码）；
- (3) 尾数加或减运算（变形补码）；
- (4) 结果规格化；
- (5) 舍入处理。

2.6.1 浮点加法、减法运算

3、规格化

(1) 规格化数的定义

即 $0.1000\dots000 \leq |M| < 1.000\dots000$

$$R = 2 \quad \frac{1}{2} \leq |M| < 1$$

(2) 规格化数的判断

$M > 0$	规格化形式
真值	$0.1 \times \times \dots \times$
原码	$0.\overset{\boxed{1}}{1} \times \times \dots \times$
补码	$\overset{\boxed{0}}{0}.\overset{\boxed{1}}{1} \times \times \dots \times$
反码	$0.1 \times \times \dots \times$

$M < 0$	规格化形式
真值	$-0.1 \times \times \dots \times$
原码	$1.\overset{\boxed{1}}{1} \times \times \dots \times$
补码	$\overset{\boxed{1}}{1}.\overset{\boxed{0}}{0} \times \times \dots \times$
反码	$1.0 \times \times \dots \times$

结论



原码：

不论正数、负数，第一数位为1



补码：

符号位和第一数位不同

2.6.1 浮点加法、减法运算

3、规格化

(3) 规格化特例

仅指补码

$$M = -\frac{1}{2} = -0.100 \dots 0$$

$$[M]_{\text{原}} = 1.100 \dots 0$$

$$[M]_{\text{补}} = \boxed{1.1}00 \dots 0$$

$\therefore [-\frac{1}{2}]_{\text{补}}$ 不是规格化的数

$$M = -1$$

$$[M]_{\text{补}} = \boxed{1.0}00 \dots 0$$

$\therefore [-1]_{\text{补}}$ 是规格化的数

为

了便于计算机硬件判断，规定：

- ◆ $-\frac{1}{2}$ 不是规格化的数
- ◆ -1 是规格化的数

(4) 左规

尾数 $\leftarrow 1$ ，阶码减 1，补码时
直到数符和第一数位不同为止。

$$\text{若 } [x+y]_{\text{补}} = 0011; 11.1001$$

$$\text{左规后 } [x+y]_{\text{补}} = 0010; 11.0010$$

$$\therefore x + y = (-0.1110) \times 2^{10}$$

(5) 右规

当 尾数溢出 (>1) 时，需 右规

即尾数出现 $01.\times\times \dots \times$ 或

$10.\times\times \dots \times$ 时

尾数 $\rightarrow 1$ ，阶码加 1

2.6.1 浮点加法、减法运算



- 练习：求 $x+y$ ，采用就近舍入，除阶符、数符外，阶码取2位，尾数有效位4位。 $x=0.1101*2^{01}$ $y=-0.1010*2^{11}$ ，

解：

(1) 对阶： $x=0.001101*2^{11}$ $y=-0.1010*2^{11}$

(2) 变形补码： 00.001101 11.0110

(3) 相加 $+11.0110$ $*2^{11}$

11.100101 (补码) $*2^{11}$

(4) 尾数规格化： 1.100101 尾数符号：1，阶码：

求真值： -0.011011 (真值) $*2^{01}$

(5) 规格化： $-1.110 *2^{01}$

阶码未采用机器码，
阶码也要变形补码

2.6.1 浮点加法、减法运算



- 练习：求 $x+y$ ，采用就近舍入，除阶符、数符外，阶码取2位，尾数有效位4位。 $x=0.1101 \times 2^{01}$ $y=-0.1010 \times 2^{11}$,

解： $[x]_{\text{补}} = 00,01; 00.1101$ $[y]_{\text{补}} = 00,11; 11.0110$

(1) 对阶： $[\Delta E]_{\text{补}} = [E_x]_{\text{补}} - [E_y]_{\text{补}} = 00,01$

$$\begin{array}{r} + \quad 11,01 \\ \hline 11,10 \end{array}$$

阶差为负（-2） $\therefore M_x \rightarrow 2 \quad E_x + 2$

$[x]_{\text{补}'} = 00,11; 00.0011$

(2) 尾数求和： $[x]_{\text{补}'} = 00.0011$

$$\begin{array}{r} + \quad [y]_{\text{补}} = 11.0110 \\ \hline 11.1001 \end{array}$$

(3) 规格化：

左规： $[x+y]_{\text{补}} = 00, 11; 11.0010$

(4) 真值： $x+y = (-0.1110) \times 2^{11}$

2.6.1 浮点加法、减法运算

- (5) 舍入处理 (对阶和向右规格化时,尾数低位丢失)
 - 就近舍入(0舍1入):类似”四舍五入”,丢弃的最高位为1,进1
 - 朝0舍入:截尾,容易累计误差
 - 朝 $+\infty$ 舍入:正数多余位不全为“0”,进1;负数,截尾
 - 朝 $-\infty$ 舍入:负数多余位不全为“0”,进1;正数,截尾
- 溢出判断和处理
 - 阶码上溢,一般将其认为是 $+\infty$ 和 $-\infty$ 。
 - 阶码下溢,则数值为0。
 - 尾数上溢 两个同符号尾数相加产生了最高位向上的进位,要将尾数右移,阶码增1来重新对齐。
 - 尾数下溢 在将尾数右移时,尾数的最低有效位从尾数域右端流出,要进行舍入处理。

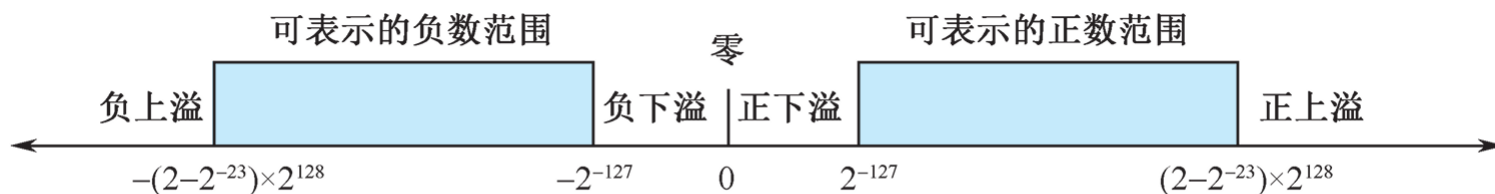


图 2.16 32 位格式浮点数的表示范围

2.6.2 浮点乘法和除法运算（了解）

- 设有两个浮点数 x 和 y :

$$x = 2^{E_x} \cdot M_x$$

$$y = 2^{E_y} \cdot M_y$$

- $x \times y = 2^{(E_x + E_y)} \cdot (M_x \times M_y)$

- $x \div y = 2^{(E_x - E_y)} \cdot (M_x \div M_y)$

- 乘除运算分为四步

- ① 0操作数检查
- ② 阶码加减操作（双符号位检测、变形补码）
- ③ 尾数乘除操作
- ④ 结果规格化
- ⑤ 舍入处理

2.6.2 浮点乘法和除法运算

- 补码采用双符号位，为了对溢出进行判断
- **00** 为正 **11** 为负
- **01** 上溢 **10** 下溢

$x = +011, y = +110$, 求 $[x + y]_{\text{补}}$ 和 $[x - y]_{\text{补}}$, 并判断是否溢出。

$$[x]_{\text{补}} = 00011, [y]_{\text{补}} = 00110, [-y]_{\text{补}} = 11010$$

$$[x + y]_{\text{补}} = [x]_{\text{补}} + [y]_{\text{补}} = 01001, \text{结果上溢。}$$

$$[x - y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}} = 11101, \text{结果正确, 为}-3。$$

2.6.2 浮点乘法和除法运算

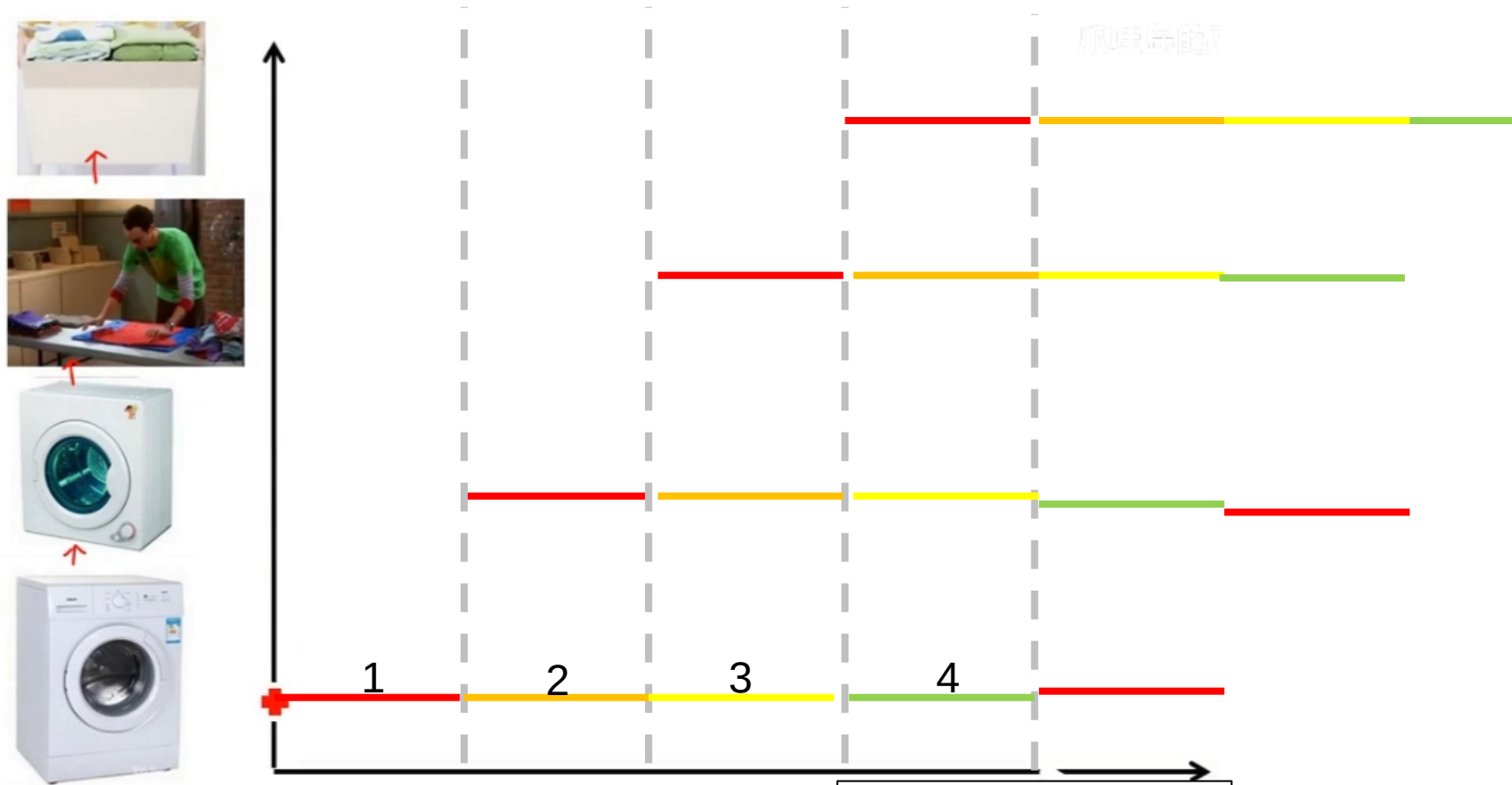
- 尾数处理
 - 截断
 - 舍入
 - 尾数用原码表示时
 - 只要尾数最低为1或者移出位中有1数值位，使最低位置1
 - 0舍1入

2.6.3 浮点运算流水线

思考：（1）什么样的例子可以用流水线做？
（2）为什么使用流水线就能提高速度？



2.6.3 浮点运算流水线



2024/3/15 思考什么任务可以用流水线完成？

任务可分解，
先后顺序（线性），
独立设备

2.6.3 浮点运算流水线

假设作业 T 被分成 k 个子任务，可表达为

$$T = \{T_1, T_2, \dots, T_k\}$$

各个子任务之间有一定的优先关系：若 $i < j$ ，则必须在 T_i 完成以后， T_j 才能开始工作。具有这种线性优先关系的流水线称为**线性流水线**。线性流水线的硬件基本结构如图 2.19 所示。

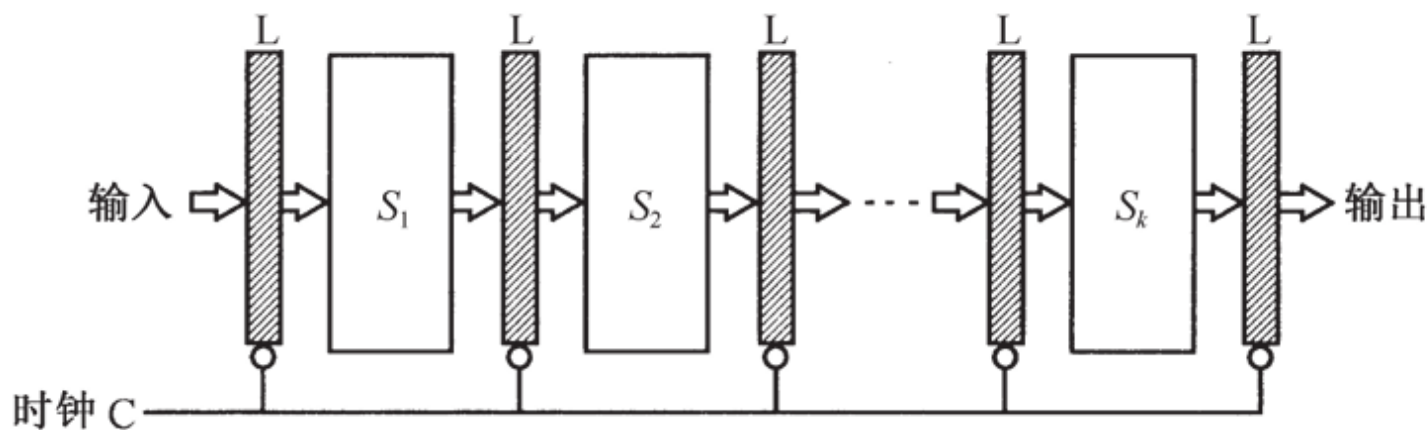


图 2.19 线性流水线的硬件基本结构

流水线浮点运算器

$$A = a \times 2^P, \quad B = b \times 2^q$$

在4级流水线加法器中实现上述浮点加法时，
分为以下操作：

- (1) 求阶差
- (2) 对阶
- (3) 相加
- (4) 规格化

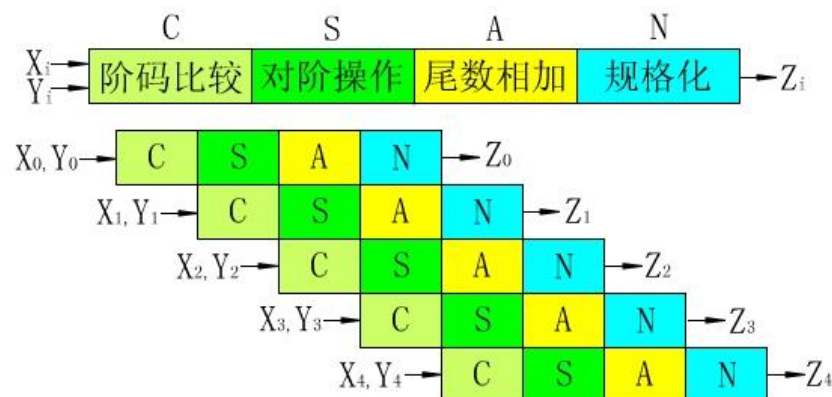
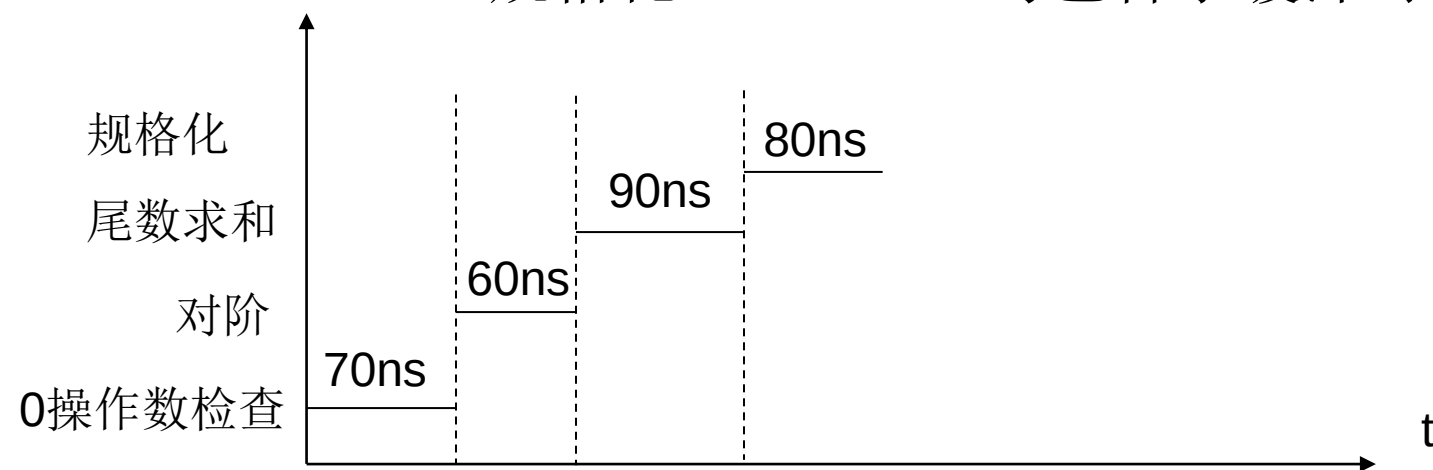


图2.21 向量加法计算的流水时空图

2.6.3 浮点运算流水线

- 假设有一个4级流水浮点加法器，每个过程段所需的时间为：0操作数检查 $\tau_1=70\text{ns}$ ，对阶 $\tau_2=60\text{ns}$ ，尾数求和 $\tau_3=90\text{ns}$ ，规格化 $\tau_4=80\text{ns}$ ，均包含了缓冲时间。



串行n个任务: $T_1 = (70+60+90+80) \times n$

并行n个任务: $T_2 = 4 \times 90 + 90 \times (n-1)$

加速比: $C = T_1/T_2$ 取极限

流水线原理

线性流水线由一系列串联的过程段组成，各个过程之间设有高速的缓冲寄存器(L)，以暂时保存上一过程子任务处理的结果。

在一个统一的时钟(C)控制下，数据从一个过程段流向相邻的过程段。设过程段 S_i 所需的时间为 t_i ，缓冲寄存器的延时为 t_l ，线性流水线的时钟周期定义为

$$\tau = \max \{ \tau_i \} + \tau_l = \tau_m + \tau_l$$

2.6.4 浮点运算器实例（了解）

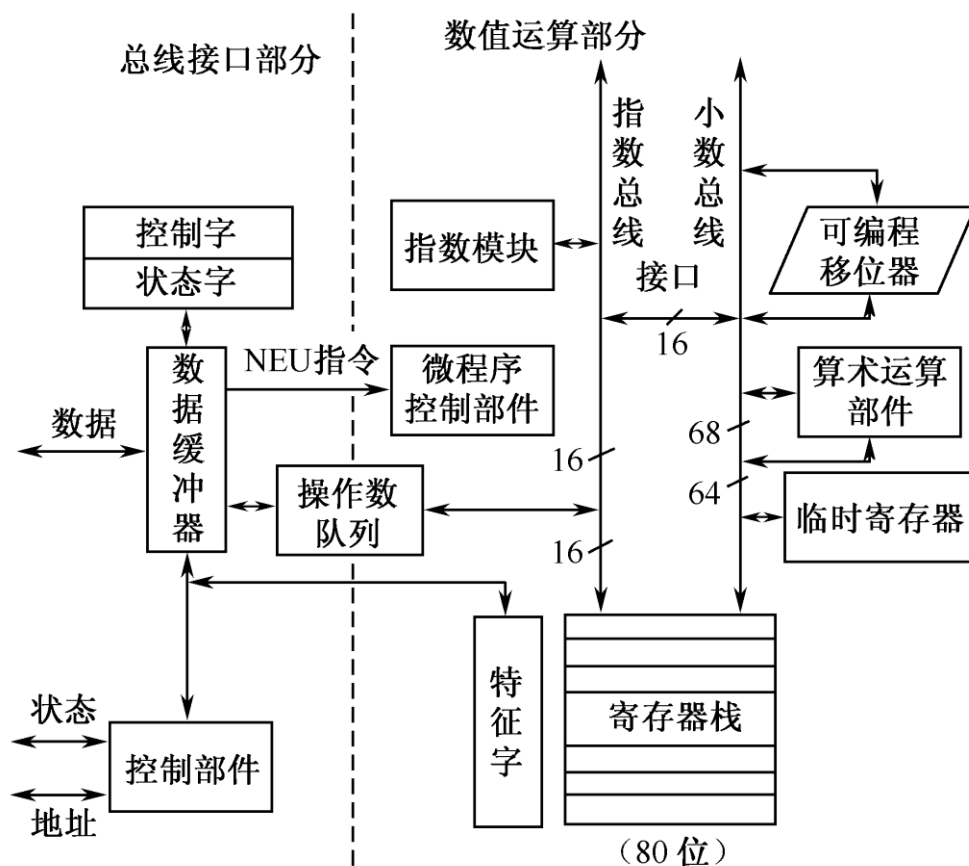


图2.23 80x87浮点运算器逻辑框图

本章小结

- 定点表示法、浮点表示法
- 按IEEE754标准规格化浮点数
- 数的真值与机器码转换：原码、反码、补码、移码
- 字符、机器码：校验
- 补码加减法，原码乘法、补码乘法。
- 运算器：串行加法器、先行进位、阵列乘法、多功能ALU、内部总线、流水线
- 浮点数加减法